A Novel Hat-Shaped Device-Cloud Collaborative Inference Framework for Large Language Models

Zuan Xie, Yang Xu, *Member, IEEE,* Hongli Xu, *Member, IEEE,* Yunming Liao, Zhiwei Yao

Abstract—Recent advancements in large language models (LLMs) have catalyzed a substantial surge in demand for LLM services. While traditional cloud-based LLM services satisfy high-accuracy requirements, they fall short in meeting critical demands for low delay and enhanced privacy. To address these limitations, we propose HAT, a novel device-cloud collaborative inference framework that leverages the complementary strengths of U-shaped inference and speculative decoding. HAT partitions the LLM into three submodels, and the input and output submodels, stacked with a lightweight adapter network, are deployed as a small language model (SLM) on each end device. Meanwhile, the middle submodel, encompassing the majority of the LLM's decoder layers, is hosted in the cloud to perform speculative decoding with on-device SLMs. During inference, HAT exchanges hidden states (rather than raw tokens) of input or draft tokens between devices and the cloud, thereby incurring substantial communication delays. Besides, processing hidden states of long prompts will exacerbate computation delays in the cloud, further compromising inference efficiency. To improve efficiency, we introduce a prompt chunking mechanism that segments long prompts into shorter chunks, enabling parallel transmission and processing. Furthermore, HAT is implemented to dynamically determine optimal chunk sizes for devices handling long prompts, thereby improving overall inference speed. Extensive experiments are conducted on a physical testbed comprising 30 NVIDIA Jetson devices and a server with 8 NVIDIA A6000 GPUs. Experimental results demonstrate that HAT achieves promising performance improvements, reducing TTFT by 41% to 54% and TBT by 41% to 77% compared to the baselines.

Index Terms—*Large Language Models, Collaborative Inference, U-Shaped Inference, Speculative Decoding, Prompt Chunking.*

1 Introduction

Recent advancements in large language models (LLMs) have revolutionized the field of natural language processing, demonstrating unprecedented capabilities across various tasks and triggering exponential growth of LLM services [1], [2]. For instance, OpenAI's ChatGPT provides various services, *e.g.*, chat-based interaction, and automated writing, to approximately 180 million users, and processes over 1.6 billion requests monthly [3]. The underlying architecture of LLM services mainly operates through an autoregressive process, which involves a *prefill* phase followed by a *decode* phase. In *prefill* phase, the LLM processes all input prompt tokens simultaneously, leveraging parallel computation to generate the initial output token. During *decode* phase, the LLM generates the rest of the content token by token, where the latest token is processed at each inference step.

Generally, the LLM services are powered by cloud servers with sufficient computing power, and can provide highly accurate solutions to complex requests [4]. Although cloud-based services satisfy the high accuracy demands of many applications, they fall short in meeting critical requirements like low delay and enhanced privacy. Specifically, the potential nature of the autoregressive process significantly reduces the inference speed, as only one token will be generated per inference step. Furthermore, directly uploading raw data to the cloud for inference exposes sensitive information during transmission or storage, and the inference output may also contain privacy-sensitive information. It is reported that over 81% of interviewed users are not satisfied with the cloud-based LLM services regarding the delay and privacy concerns, which are essential for some popular applications, such as autonomous driving and health monitoring [5]–[7].

As the computing capabilities of end devices increase, many studies propose device-cloud collaborative inference frameworks to address the two concerns in cloud-based LLM services [8]. We summarize the advantages and disadvantages of different inference frameworks in Table 1. Concretely, a basic collaborative inference framework (as adopted in Apple Intelligence [9]) is proposed to deploy small language models (SLMs) on end devices to handle some simple delay-sensitive or privacy-sensitive requests locally, while other complex requests are offloaded to the in-cloud LLMs for processing. With local SLMs, this framework can improve the inference speed and alleviate the privacy issue to a certain extent. However, the limited performance of SLMs may result in reduced inference accuracy and user experience.

In order to maintain satisfactory inference accuracy while improving overall inference speed, speculative decoding (SD) has been integrated into the device-cloud inference

• *Z. Xie, Y. Xu, H. Xu, Y. Liao and Z. Yao are with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, Anhui, China, 230027, and also with Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou, Jiangsu, China, 215123. E-mails: xz1314@mail.ustc.edu.cn; xuyangcs@ustc.edu.cn; xuhongli@ustc.edu.cn; ymliao98@mail.ustc.edu.cn; zhiweiyao@mail.ustc.edu.cn.*

一种新型的帽子型大语言模型 端云协同推理框架

谢钻、徐阳、*Member, IEEE*,徐红利、*Member, IEEE*,廖云明、姚志伟

摘要：大语言模型 (LLM) 的最新进展促进了 LLM 服务需求的大幅增长。虽然传统的基于云的大语言模型服务可以满足高精度要求，但它们无法满足低延迟和增强隐私的关键需求。为了解决这些限制，我们提出了 HAT，这是一种新颖的设备-云协作推理框架，它利用了 U 形推理和推测解码的互补优势。 HAT 将 LLM 划分为三个子模型，输入和输出子模型与轻量级适配器网络堆叠在一起，作为小语言模型 (SLM) 部署在每个终端设备上。同时，中间子模型包含了 LLM 解码器的大部分层，托管在云端，以便使用设备上的 SLM 执行推测解码。在推理过程中，HAT 在设备和云之间交换输入或草稿令牌的隐藏状态（而不是原始令牌），从而导致严重的通信延迟。此外，处理长提示的隐藏状态会加剧云端的计算延迟，进一步降低推理效率。为了提高效率，我们引入了提示分块机制，将长提示分割成较短的块，从而实现并行传输和处理。此外，HAT 的实现是为了动态确定处理长提示的设备的最佳块大小，从而提高整体推理速度。在由 30 个 NVIDIA Jetson 设备和一台配备 8 个 NVIDIA A6000 GPU 的服务器组成的物理测试台上进行了大量实验。实验结果表明，HAT 实现了令人鼓舞的性能改进，与基线相比，TTFT 减少了 41% 至 54%，TBT 减少了 41% 至 77%。

索引术语—*Large Language Models, Collaborative Inference, U-Shaped Inference, Speculative Decoding, Prompt Chunking.*

1 简介

大语言模型 (LLM) 的最新进展彻底改变了自然语言处理领域，展示了跨各种任务的前所未有的能力，并引发了 LLM 服务的指数级增长 [1]、[2]。例如，OpenAI 的 ChatGPT 为大约 1.8 亿用户提供各种服务、*e.g.*、基于聊天的交互和自动写作，每月处理超过 16 亿个请求 [3]。 LLM服务的底层架构主要通过自回归过程运行，其中涉及*prefill*阶段和随后的*decode*阶段。在*prefill*阶段，LLM同时处理所有输入提示令牌，利用并行计算生成初始输出令牌。在 *decode* 阶段，LLM 逐个生成内容令牌的其余部分，其中在每个推理步骤中处理最新的令牌。

一般来说，LLM服务由具有足够计算能力的云服务器提供支持，可以为复杂的请求提供高精度的解决方案[4]。尽管基于云的服务满足了许多应用程序的高精度需求，但它们无法满足关键要求：低延迟和增强隐私等评论。具体来说，自回归过程的潜在性质显着降低了推理速度，因为每个推理步骤仅生成一个令牌。此外，直接将原始数据上传到云端进行推理会在传输或存储过程中暴露敏感信息，推理输出也可能包含隐私敏感信息。据报道，超过 81% 的受访用户对基于云的 LLM 服务不满意，因为延迟和隐私问题对一些热门应用至关重要，例如自动驾驶和健康监测 [5]-[7]。

随着终端设备计算能力的增强，许多研究提出了端云协同推理框架来解决基于云的LLM服务中的两个问题[8]。我们在表1中总结了不同推理框架的优缺点。具体来说，提出了一个基本的协作推理框架（如Apple Intelligence [9]中采用的），在终端设备上部署小语言模型（SLM），以在本地处理一些简单的延迟敏感或隐私敏感请求，而其他复杂请求则卸载到云中LLM进行处理。通过本地SLM，该框架可以提高推理速度并在一定程度上缓解隐私问题。然而，SLM 的性能有限可能会导致推理精度和用户体验降低。

为了保持令人满意的推理精度，同时提高整体推理速度，推测解码（SD）已集成到端云推理中

- *Z. Xie, Y. Xu, H. Xu, Y. Liao and Z. Yao are with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, Anhui, China, 230027, and also with Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou, Jiangsu, China, 215123. E-mails: xz1314@mail.ustc.edu.cn; xuyangcs@ustc.edu.cn; xuhongli@ustc.edu.cn; ymliao98@mail.ustc.edu.cn; zhiweiyao@mail.ustc.edu.cn.*

frameworks [10]–[12]. Following the idea of SD, the SLM on an end device generates a multi-token draft sequence, which undergoes parallel verification by the in-cloud LLM with a single inference step (also called a verification step). The parallel verification significantly accelerates inference compared to traditional sequential generation while preserving high accuracy through the rejection of divergent draft tokens and subsequent content. However, as the input and output tokens are shared between devices and the cloud, this framework still inevitably raises significant privacy concerns.

To enhance privacy, split inference has been proposed as a privacy-preserving device-cloud inference framework [13]–[15]. This framework splits an LLM into two submodels and deploys them separately on end devices and the cloud. Then, LLM inference is performed in a pipeline fashion, where the intermediate data (*i.e.*, hidden states) of each on-device submodel (rather than raw tokens) are uploaded to the cloud to derive output tokens. To further protect the privacy of sensitive outputs, a variant of split inference, called U-shaped inference, partitions an LLM into three submodels [16]. The input and output submodels are deployed on devices, while the computationally intensive middle submodel, encompassing the majority of transformer layers, resides in the cloud. Although this framework effectively safeguards both input and output tokens from exposure beyond devices, it incurs considerable communication overhead for the exchange of hidden states, which are much larger than tokens [17]. Moreover, during *decode* phase, the iterative generation process necessitates frequent transmission of hidden states between devices and the cloud, reducing decoding efficiency.

As far as we know, existing device-cloud inference frameworks have struggled to concurrently optimize delay, privacy, and accuracy. By leveraging the complementary strengths of U-shaped inference (for privacy) and speculative decoding (for delay), we propose a novel device-cloud collaborative inference framework termed *HAT*. Similar to the U-shaped framework, HAT deploys the LLM's input and output submodels on devices to protect privacy. Besides, a lightweight adapter network is constructed via knowledge distillation and combined with the LLM's input and output submodels to form an SLM, called draft model. The on-device draft model performs speculative decoding with the LLM, effectively reducing the frequency of device-cloud communication during the inference procedure and maintaining promising accuracy. However, in HAT, devices with long prompts will suffer from significant communication delays when transmitting all hidden states at once during *prefill* phase. Moreover, since the hidden states of devices operating in different phases (*prefill* and *decode*) are processed in batch by the LLM simultaneously at each inference step, the devices in *decode* phase will experience prolonged in-cloud computation delays when their hidden states are batched with those from the devices with long prompts in *prefill* phase.

To address the degradation of communication and computational efficiencies caused by long prompts, we propose parallelizing the transmission and processing of hidden states through prompt chunking. During *prefill* phase, HAT divides long prompts into multiple chunked prompts, enabling the overlapping of device-side transmission of chunked hidden states with in-cloud computation of previously adjacent chunked hidden states, thereby effectively reducing bulk communication delays. Moreover, prompt chunking also alleviates in-cloud computation delays for devices in *decode* phase, as their hidden states are batched with much shorter chunked hidden states from devices in *prefill* phase. However, excessively large chunks diminish the benefits of parallelization, while overly small chunks inversely lead to increased computation delays for devices in *prefill* phase due to fragmentation, which necessitates optimizing the size of chunks in HAT. Our main contributions are as follows:

TABLE 1: Advantages and Disadvantages of Different Device-Cloud Collaborative Inference Frameworks.

Metric	Basic	SD	U-shape	Ours
Delay	◐	●	○	●
Privacy	◐	○	●	●
Accuracy	◐	●	●	●

- We introduce a novel device-cloud collaborative inference framework, termed HAT, that integrates U-shaped inference with speculative decoding to deliver LLM services with promising accuracy, low latency, and enhanced privacy.
- We propose a prompt chunking mechanism to enhance the communication and computational efficiencies of HAT, especially in scenarios involving long prompts. As the size of prompt chunks has a comprehensive influence on the processing of hidden states for devices in different phases during inference. Thus, HAT is designed to dynamically determine optimal chunk sizes for devices handling long prompts, thereby accelerating LLM inference.
- The performance of HAT is evaluated through a physical platform with a total of 30 NVIDIA Jetson devices. The experimental results show that HAT can reduce the time-to-first-token (TTFT) by 41% to 54%, while reducing the time-between-tokens (TBT) by 41% to 77%, compared to the baselines.

2 Background and Motivation

2.1 General Inference Process of LLMs

Autoregressive Inference for A Single Request. Popular LLMs, such as GPT-3 [18] and LLaMa [19], are decoder-only models, which are built from a stack of decoder layers with identical structures, each consisting of two core components: a self-attention module and a feed-forward neural network [17]. The self-attention module computes context-aware weights for token interactions, allowing the model to capture and aggregate semantic relationships across the entire sequence effectively. Subsequently, the feed-forward network processes these relationships to generate updated hidden states, which are then propagated to the next layer for further refinement.

The inference process of LLMs mainly operates in an autoregressive manner, and consists of two distinct phases: *prefill* phase and *decode* phase. In *prefill* phase, the LLM processes all tokens of a request's input prompt in parallel to produce the first output token. Following this, *decode* phase begins to generate the remaining tokens sequentially with one token at a time. Concretely, at each inference

框架[10]-[12]。遵循 SD 的思想，终端设备上的 SLM 生成多令牌草稿序列，该序列由云中的 LLM 通过单个推理步骤（也称为验证步骤）进行并行验证。与传统的顺序生成相比，并行验证显着加速了推理，同时通过拒绝不同的草稿令牌和后续内容来保持高精度。然而，由于输入和输出令牌在设备和云之间共享，该框架仍然不可避免地引起严重的隐私问题。

为了增强隐私，分割推理被提出作为隐私保护的设备云推理框架[13]-[15]。该框架将LLM分成两个子模型，并将它们分别部署在终端设备和云端。然后，LLM 推理以管道方式执行，其中每个设备上子模型（而不是原始令牌）的中间数据（*i.e.*，隐藏状态）被上传到云端以导出输出令牌。为了进一步保护敏感输出的隐私，一种称为 U 形推理的分割推理变体将 LLM 划分为三个子模型 [16]。输入和输出子模型部署在设备上，而包含大部分变压器层的计算密集型中间子模型则驻留在云中。尽管这个框架有效地保护了输入和输出令牌免于暴露在设备之外，但它会在交换隐藏状态时产生相当大的通信开销，这些开销比令牌大得多[17]。此外，在*decode*阶段，迭代生成过程需要在设备和云端之间频繁传输隐藏状态，从而降低了解码效率。

据我们所知，现有的端云推理框架一直在努力同时优化延迟、隐私和准确性。通过利用 U 形推理（针对隐私）和推测性解码（针对延迟）的互补优势，我们提出了一种新颖的设备-云协作推理框架，称为 *HAT*。与U型框架类似，HAT在设备上部署LLM的输入和输出子模型以保护隐私。此外，通过知识蒸馏构建轻量级适配器网络，并与LLM的输入和输出子模型结合形成SLM，称为草案模型。设备上的草稿模型通过 LLM 进行推测性解码，有效降低了推理过程中设备与云的通信频率，并保持了良好的准确性。然而，在 HAT 中，在 *prefill* 阶段一次传输所有隐藏状态时，具有长提示的设备将遭受严重的通信延迟。此外，由于LLM在每个推理步骤同时对不同阶段（*prefill*和*decode*）运行的设备的隐藏状态进行批量处理，因此当其隐藏状态与*prefill*阶段中长提示的设备的隐藏状态进行批处理时，*decode*阶段的设备将经历长时间的云端计算延迟。

为了解决长提示导致的通信和计算效率下降的问题，我们建议通过提示分块并行化隐藏状态的传输和处理。在 *prefill* 阶段，HAT 将长提示分为多个分块提示，en-

表 1：不同端云协同推理框架的优缺点。

Metric	Basic	SD	U-shape	Ours
Delay	◐	●	○	●
Privacy	◐	○	●	●
Accuracy	◐	●	●	●

能够将分块隐藏状态的设备端传输与先前相邻分块隐藏状态的云计算重叠，从而有效减少批量通信延迟。此外，即时分块还可以减轻 *decode* 阶段设备的云内计算延迟，因为它们的隐藏状态是与 *prefill* 阶段设备的更短的分块隐藏状态进行批处理的。然而，过大的块会削弱并行化的好处，而过小的块则会因碎片而导致设备在*prefill*阶段的计算延迟增加，这就需要优化HAT中的块的大小。我们的主要贡献如下：

- 我们引入了一种新颖的设备-云协作推理框架，称为 HAT，它将 U 形推理与推测解码相集成，以提供具有良好准确性、低延迟和增强隐私性的 LLM 服务。
- 我们提出了一种提示分块机制来提高 HAT 的通信和计算效率，特别是在涉及长提示的场景中。由于提示块的大小对推理过程中不同阶段的设备隐藏状态的处理有综合影响。因此，HAT 旨在动态确定处理长提示的设备的最佳块大小，从而加速 LLM 推理。
- HAT 的性能通过总共 30 个 NVIDIA Jetson 设备的物理平台进行评估。实验结果表明，与基线相比，HAT 可以将首次令牌时间 (TTFT) 减少 41% 至 54%，同时将令牌间隔时间 (TBT) 减少 41% 至 77%。

2 背景和动机

2.1 LLM的一般推理过程

单个请求的自回归推理。流行的LLM，例如GPT-3 [18]和LLaMa [19]，是纯解码器模型，它们是由具有相同结构的解码器层堆栈构建的，每个解码器层由两个核心组件组成：自注意力模块和前馈神经网络[17]。自注意力模块计算令牌交互的上下文感知权重，使模型能够有效地捕获和聚合整个序列的语义关系。随后，前馈网络处理这些关系以生成更新的隐藏状态，然后将其传播到下一层以进一步细化。

LLM的推理过程主要以自回归方式运行，由两个不同的阶段组成：*prefill*阶段和*decode*阶段。在 *prefill* 阶段，LLM 并行处理请求输入提示的所有标记，以生成第一个输出标记。此后，*decode* 阶段开始依次生成剩余令牌，一次一个令牌。具体来说，在每次推理时

step, the LLM predicts the next token for the request based on its input prompt and all previously generated tokens. This process continues iteratively until an end-of-sequence (EOS) token is produced or the maximum token limit is reached. A key requirement of *decode* phase is access to the keys and values of all previously processed tokens for attention computations. To avoid redundant calculations, modern LLM systems utilize a key-value (KV) cache to store precomputed keys and values, allowing for efficient attention computations during the generation of subsequent tokens.

Batched Inference for Multiple Requests. Usually, commercial LLM systems are designed to handle multiple concurrent requests from various devices. Processing these requests sequentially will result in low throughput, as GPU resources often remain underutilized while waiting for individual requests to complete. To improve the throughput, many LLM systems adopt continuous batching [20], [21] to process multiple requests within a single batch, which is called *Batched Inference*. It allows requests to dynamically join or leave the batch after completing each inference step, eliminating the need to wait for all requests in the batch to finish their final generation.

The computational resources for batched inference primarily depend on the total number of tokens in the batch [21] Since the KV-cache stores the results of all previously processed tokens, the token size for *decode* phase is effectively 1, whereas the token size for *prefill* phase corresponds to the length of the input prompt. As a result, batching is highly efficient for *decode* phase, as it significantly improves GPU utilization with minimal delay. However, when processing requests from different phases (*i.e.*, *prefill* and *decode*) within the same batch, requests with long prompts in *prefill* phase always consume a disproportionate amount of computational resources. This imbalance leads to increased computation delays for other requests in *decode* phase. To mitigate the issue, recent studies [21], [22] propose to divide the prompts of devices in *prefill* phase into many shorter chunks. These chunks are processed across multiple inference steps, where the first output token of each device is generated until all its prompt chunks are fed to the LLM. The detailed evaluation is presented in Sec. 2.3.

2.2 Collaborative Inference

Our framework integrates the distinct strengths of U-shaped inference and speculative decoding to simultaneously enhance inference latency, data privacy, and accuracy for LLMs. Below, we provide a detailed explanation of the two collaborative inference frameworks.

Speculative Decoding. The autoregressive nature of LLMs inherently constrains their inference speed, particularly in cloud-centric LLM systems. To mitigate this limitation, speculative decoding has been incorporated in the device-cloud collaborative inference framework that employs both an SLM and an LLM for token drafting and verification, respectively [10], [11]. Each round of speculative decoding consists of two stages: drafting and verification. In the drafting stage, the SLM, typically deployed on end devices, generates a sequence of multiple tokens through autoregressive processing. Subsequently, during the verification stage, this draft sequence is fed to the in-cloud LLM for verification through a single inference step. The verification process involves comparing the draft tokens with the LLM's output, where matched tokens are accepted. Notably, the LLM's inference result following the last accepted draft token serves as the input for the subsequent round of speculative decoding. While SLMs may not produce entirely accurate or comprehensive sequences, they demonstrate proficiency in generating simpler linguistic elements such as qualifiers, pronouns, and punctuation marks, which exhibit higher acceptance rates during verification. When the draft sequence achieves sufficient accuracy, speculative decoding enables the LLM to produce multiple tokens in a single inference step, thereby substantially reducing the reliance on autoregressive inference and significantly enhancing overall inference speed.

The effectiveness of speculative decoding hinges on the alignment of output distributions between the SLM and the LLM. To achieve this, some approaches [23] use models from the same family but of different sizes as the SLM and LLM (*e.g.*, Vicuna-68M *vs.* Vicuna-7B). However, those approaches impose constraints on model selection, limiting their flexibility. Therefore, alternative approaches [24] involve training lightweight, custom-designed models on small public datasets to serve as SLMs. Furthermore, recent advancements [25] have introduced tree-based verification methods to optimize the number of accepted draft tokens at a single inference step. Concretely, multiple candidate draft tokens at different positions are combined into a tree-like structure, where each branching sequence represents a potential path that may be accepted in the verification stage.

U-shaped Inference. To safeguard the privacy of both input and output tokens on local devices, U-shaped inference has been proposed, which partitions the LLM into three submodels. The lightweight input and output submodels are deployed on end devices, while the computationally intensive middle submodel operates in the cloud. During inference, input tokens are first encoded into "shallow" hidden states by the local input submodel on each device. Subsequently, these hidden states are transmitted to the cloud for further processing by the middle submodel. Finally, the refined "deep" hidden states are transferred back to the device, where the output submodel decodes them into final predicted tokens.

U-shaped inference ensures raw input/output tokens remain confined to local devices, which enhances the privacy of raw data. However, it introduces significant communication overhead, as each token generation requires bidirectional transmission of hidden states. This overhead is exacerbated during *prefill* phase, where long prompts produce large volumes of hidden states, causing severe communication delays.

2.3 Motivation

To investigate the efficiencies of different inference frameworks, as well as the impact of long prompts on communication and in-cloud computation delays, we conduct preliminary experiments with the Vicuna model [26] on the Specbench dataset [27]. In the experiments, a deep learning workstation equipped with 8 NVIDIA A6000 GPUs serves as the cloud server, and 3 NVIDIA Jetson AGX Orin devices

步骤，LLM 根据其输入提示和所有先前生成的令牌来预测请求的下一个令牌。此过程不断迭代，直到生成序列结束 (EOS) 令牌或达到最大令牌限制。 *decode* 阶段的一个关键要求是访问所有先前处理的令牌的键和值以进行注意力计算。为了避免冗余计算，现代 LLM 系统利用键值 (KV) 缓存来存储预先计算的键和值，从而在后续令牌的生成过程中实现高效的注意力计算。

多个请求的批量推理。通常，商业 LLM 系统旨在处理来自各种设备的多个并发请求。按顺序处理这些请求将导致吞吐量较低，因为在等待单个请求完成时 GPU 资源通常仍未得到充分利用。为了提高吞吐量，许多LLM系统采用连续批处理[20]、[21]在单个批次内处理多个请求，称为 *Batched Inference*。它允许请求在完成每个推理步骤后动态加入或离开批次，从而无需等待批次中的所有请求完成最终生成。

批量推理的计算资源主要取决于批次中的令牌总数[21]由于KV缓存存储了所有先前处理的令牌的结果，因此*decode*阶段的令牌大小实际上是1，而*prefill*阶段的令牌大小对应于输入提示的长度。因此，批处理对于 *decode* 阶段非常高效，因为它以最小的延迟显着提高了 GPU 利用率。然而，当处理同一批次中不同阶段（*i.e.*、*prefill*和*decode*）的请求时，*prefill*阶段中带有长提示的请求总是会消耗不成比例的计算资源。这种不平衡会导致 *decode* 阶段其他请求的计算延迟增加。为了缓解这个问题，最近的研究[21]、[22]建议将*prefill*阶段设备的提示分成许多较短的块。这些块通过多个推理步骤进行处理，其中生成每个设备的第一个输出令牌，直到其所有提示块都馈送到 LLM。详细的评估在第 2 节中介绍。 2.3.

2.2 协同推理

我们的框架集成了 U 形推理和推测解码的独特优势，同时增强了大语言模型的推理延迟、数据隐私和准确性。下面，我们对这两个协作推理框架进行详细解释。

推测性解码。大语言模型的自回归本质本质上限制了它们的推理速度，特别是在以云为中心的大语言模型系统中。为了减轻这一限制，推测性解码已被纳入设备-云协作推理框架中，该框架分别采用 SLM 和 LLM 进行令牌起草和验证[10]、[11]。每轮推测解码由两个阶段组成：起草和验证。在起草阶段，SLM（通常部署在终端设备上）通过自回归处理生成多个令牌的序列。随后，在验证阶段，该草稿序列将被馈送到云端 LLM 通过单个推理步骤进行验证。验证过程涉及将草案令牌与法学硕士的输出进行比较，其中匹配的令牌被接受。值得注意的是，LLM 在最后接受的草稿令牌之后的推理结果将作为下一轮推测解码的输入。虽然 SLM 可能无法生成完全准确或全面的序列，但它们在生成更简单的语言元素（例如限定词、代词和标点符号）方面表现出熟练程度，这些元素在验证过程中表现出更高的接受率。当草稿序列达到足够的精度时，推测性解码使 LLM 能够在单个推理步骤中生成多个标记，从而大大减少对自回归推理的依赖，并显着提高整体推理速度。

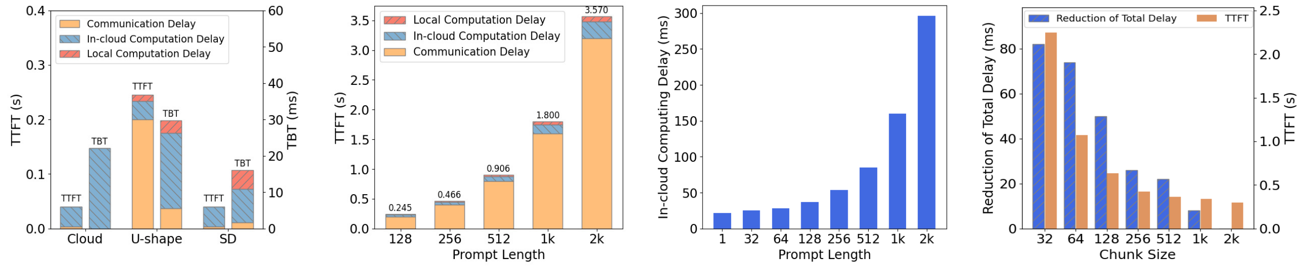
推测性解码的有效性取决于 SLM 和 LLM 之间输出分布的对齐。为了实现这一目标，一些方法 [23] 使用与 SLM 和 LLM 相同但大小不同的模型（*e.g.*、Vicuna-68M *vs.* Vicuna-7B ）。然而，这些方法对模型选择施加了限制，限制了它们的灵活性。因此，替代方法 [24] 涉及在小型公共数据集上训练轻量级、定制设计的模型以充当 SLM。此外，最近的进展 [25] 引入了基于树的验证方法，以优化单个推理步骤中接受的草稿令牌的数量。具体而言，将不同位置的多个候选草案令牌组合成树状结构，其中每个分支序列代表验证阶段可以接受的潜在路径。

U 形推理。为了保护本地设备上输入和输出令牌的隐私，提出了 U 形推理，它将 LLM 分为三个子模型。轻量级的输入和输出子模型部署在终端设备上，而计算密集型的中间子模型则在云端运行。在推理过程中，输入标记首先由每个设备上的本地输入子模型编码为“浅”隐藏状态。随后，这些隐藏状态被传输到云端以供中间子模型进一步处理。最后，精炼的“深层”隐藏状态被传输回设备，输出子模型将它们解码为最终的预测标记。

U 形推理确保原始输入/输出令牌仍然局限于本地设备，从而增强了原始数据的隐私性。然而，它引入了显着的通信开销，因为每个令牌生成都需要隐藏状态的双向传输。这种开销在 *prefill* 阶段会加剧，其中长提示会产生大量隐藏状态，从而导致严重的通信延迟。

2.3 动机

为了研究不同推理框架的效率，以及长提示对通信和云计算延迟的影响，我们在 Specbench 数据集 [27] 上使用 Vicuna 模型 [26] 进行了初步实验。在实验中，配备8个NVIDIA A6000 GPU的深度学习工作站作为云服务器，以及3个NVIDIA Jetson AGX Orin设备



(a) Delay of different inference frameworks (b) TTFT of U-shaped inference with different prompt lengths (c) In-cloud computation delay with different prompt lengths (d) Effect of prompt chunking with different chunk sizes

Fig. 1: Results of Preliminary Experiments.

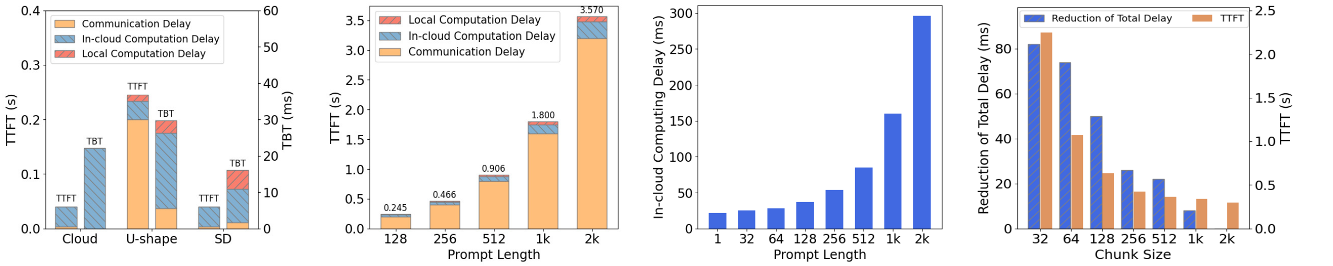
are adopted as end devices. For speculative decoding (SD), we separately deploy Vicuna-7B and Vicuna-68M on the server (for verification) and a device (for drafting). Besides, for U-shaped inference, we deploy the first 2 decoder layers and the last head of Vicuna-7B on a device, and the remaining portion on the server. As a comparison, we directly forward the request of a device to the server-side Vicuna-7B for inference as the cloud-based baseline.

Delay of Different Inference Frameworks. There are two primary metrics for evaluating LLM inference speed, *i.e.*, time-to-first-token (TTFT) and time-between-tokens (TBT). TTFT measures the delay of generating the first output token from the moment a request arrives in *prefill* phase, while TBT measures the time interval between the generation of consecutive output tokens during *decode* phase. Firstly, we illustrate the TTFT and the TBT (involving local computation delay, communication delay, and in-cloud computation delay) of different inference frameworks to process a request with the 128-token prompt in Figure 1(a), in which the proportion of computation delay and communication delay varies significantly among inference frameworks. On one hand, SD makes full use of the local computational resources of devices to generate a multi-token draft sequence, which undergoes parallel verification by the in-cloud LLM, reducing the in-cloud computation delay and accelerating inference, compared to cloud-based inference. On the other hand, since the size of hidden states is much larger than that of tokens, the communication delay for transmitting hidden states in U-shaped inference becomes substantial, especially in *prefill* phase, leading to higher TTFT and TBT compared to both SD and cloud-based inference.

Impact of Long Prompts on Communication. As shown in Figure 1(a), the communication delay in U-shaped inference accounts for more than 80% of the TTFT in *prefill* phase with the 128-token prompt. We further explore the impact of varing prompt lengths (ranging from 128 to 2k tokens) on TTFT and communication delay in U-shaped inference. By the results in Figure 1(b), the communication delay increases almost linearly with the prompt length increasing and always occupies the majority of the TTFT. Specifically, the communication delay of the 512-token prompt is 4× that of the 128-token prompt. Besides, for the long prompts with 2k tokens, the TTFT is 3.57s, while the delays of local computation, in-cloud computation, and communication are 0.09s (2.5%), 0.28s (7.8%), and 3.20s (89.6%), respectively. Consequently, optimizing the handling of long prompts can yield substantial improvements in reducing communication delays for U-shaped inference and our system design.

Impact of Long Prompts on In-Cloud Computation. In practice, in-cloud LLMs usually process requests from different phases (*prefill* and *decode*) simultaneously through batched inference. According to our analysis in Section 2.1, requests with long prompts in *prefill* phase can consume excessive computational resources and lead to increased in-cloud computation delay for other requests in *decode* phase. Therefore, we let the LLM process the batch consisting of one request with varying prompt lengths in *prefill* phase and nine requests in *decode* phase to explore the impact of prompt length on the in-cloud computation delay. By Figure 1(c), as the prompt length increases, the in-cloud computation delay becomes higher and higher. For instance, the in-cloud computation delay caused by 32-token prompt length is only 10.1% higher than that caused by 1-token prompt length, while for prompt length more than 512 tokens, in-cloud computation delay almost increases linearly with the prompt length. That is because batching *prefill* phase with short prompts would improve GPU utilization with minimal delay increase, while long prompts would lead to saturation of computational resources and prolonged delays.

Effect of Prompt Chunking. To reduce the in-cloud computation delay caused by long prompts, an intuitive solution is to divide long prompts into multiple chunked prompts (*i.e.*, prompt chunking), which are processed separately across multiple inference steps. To evaluate the effect of prompt chunking, we perform the consecutive 64-step batched inference by dividing a 2k-token prompt into multiple chunked prompts, in which the previous batches consist of one chunked prompt in *prefill* phase as well as nine requests in *decode*, and the remaining batches after processing the entire prompt only consist of ten requests in *decode*. The results are presented in Figure 1(d), in which we record the reduction of total computation delay (*i.e.*, the in-cloud computation delay for the consecutive 64-step batched inference) and the TTFT after processing all chunks in *prefill* phase given different chunk sizes. As evidenced by the results in Figure 1(d), a decrease in chunk size correlates with an increase in the reduction of total computation delay and TTFT; however, the TTFT escalates at a faster rate. For example, with a chunk size of 32, the total delay is reduced by 82.3 ms, but the TTFT surges by approximately 6.6× compared to inference without prompt chunking. In summary, while prompt chunking effectively alleviates in-



(a) 不同推理框架的延迟 (b) 不同提示长度的 U 形推理的 TTFT (c) 不同提示长度的云内计算延迟 (d) 提示分块机智的效果 小时
不同的块大小

图 1：初步实验结果。

被采用作为终端设备。对于推测解码（SD），我们分别在服务器（用于验证）和设备（用于起草）上部署Vicuna-7B和Vicuna-68M。此外，对于 U 形推理，我们将 Vicuna-7B 的前 2 个解码器层和最后一个头部署在设备上，其余部分部署在服务器上。作为比较，我们直接将设备的请求转发到服务器端 Vicuna-7B 进行推理，作为基于云的基线。

不同推理框架的延迟。评估 LLM 推理速度有两个主要指标：*i.e.*、首次令牌时间 (TTFT) 和令牌间隔时间 (TBT)。 TTFT 测量从请求到达 *pre- fill* 阶段开始生成第一个输出令牌的延迟，而 TBT 测量在 *decode* 阶段生成连续输出令牌之间的时间间隔。首先，我们展示了不同推理框架处理图1（a）中128个token提示的请求的TTFT和TBT（包括本地计算延迟、通信延迟和云计算延迟），其中计算延迟和通信延迟的比例在不同推理框架之间存在显着差异。一方面，SD充分利用设备本地计算资源生成多token草稿序列，并由云端LLM并行验证，与云端推理相比，减少了云端计算延迟，加速了推理速度。另一方面，由于隐藏状态的大小远大于令牌的大小，因此 U 形推理中传输隐藏状态的通信延迟变得很大，尤其是在 *prefill* 阶段，导致与 SD 和基于云的推理相比更高的 TTFT 和 TBT 。

长提示对沟通的影响。如图1（a）所示，U形推理中的通信延迟占128个token提示的*prefill*阶段TTFT的80%以上。我们进一步探讨了不同提示长度（范围从 128 到 2k 标记）对 TTFT 和 U 形推理中通信延迟的影响。从图1（b）的结果可以看出，通信延迟随着提示长度的增加几乎呈线性增加，并且始终占据TTFT的大部分。具体来说，512个令牌提示的通信延迟是128个令牌提示的4×。此外，对于2k代币的长提示，TTFT为3.57s，而本地计算、云端计算和通信的延迟分别为0.09s（2.5%）、0.28s（7.8%）和3.20s（89.6%）。因此，优化长提示的处理可以在减少 U 形推理和我们的系统设计的通信延迟方面产生了重大改进。

长提示对云计算的影响。在实践中，云端大语言模型通常通过批量推理同时处理来自不同阶段（*prefill*和*decode*）的请求。根据我们在2.1节中的分析，*prefill*阶段长提示的请求会消耗过多的计算资源，并导致*decode*阶段其他请求的云内计算延迟增加。因此，我们让LLM在*prefill*阶段处理由一个具有不同提示长度的请求和在*decode*阶段的九个请求组成的批次，以探讨提示长度对云计算延迟的影响。由图1(c)可知，随着提示长度的增加，云内计算延迟变得越来越高。例如，32个token提示长度引起的云内计算延迟仅比1个token提示长度引起的云计算延迟高10.1%，而对于超过512个token的提示长度，云内计算延迟几乎随提示长度线性增加。这是因为具有短提示的批处理 *prefill* 阶段会以最小的延迟增加提高 GPU 利用率，而长提示会导致计算资源饱和和延长延迟。

即时分块的效果。为了减少长提示引起的云端计算延迟，一种直观的解决方案是将长提示分为多个分块提示（*i.e.*，提示分块），这些提示在多个推理步骤中单独处理。为了评估提示分块的效果，我们通过将 2k-token 提示分为多个分块提示来执行连续 64 步批量推理，其中前一批包含 *prefill* 阶段的一个分块提示以及 *decode* 中的 9 个请求，处理整个提示后的剩余批次仅包含 *decode* 中的 10 个请求。结果如图 1(d) 所示，其中我们记录了总计算延迟的减少（*i.e.*，连续 64 步批量推理的云内计算延迟）以及在给定不同块大小的情况下处理 *prefill* 阶段中的所有块后的 TTFT。如图 1(d) 中的结果所证明，块大小的减小与总计算延迟和 TTFT 的减少的增加相关；然而，TTFT 升级的速度更快。例如，块大小为 32 时，总延迟减少了 82.3 毫秒，但与没有提示分块的推理相比，TTFT 激增了约 6.6×。总之，虽然及时分块有效地缓解了 -

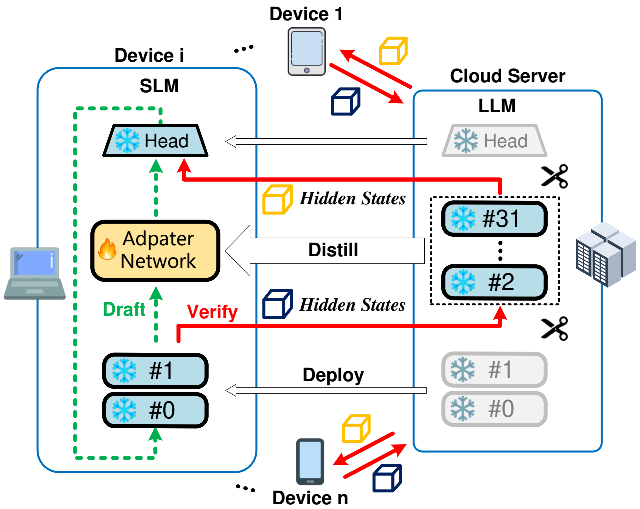


Fig. 2: Illustration of HAT.

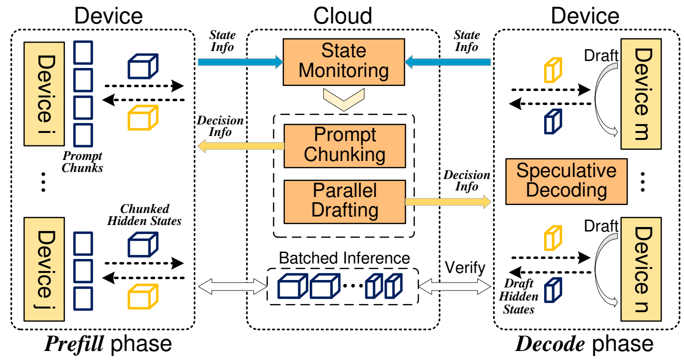


Fig. 3: System overview of HAT.

cloud computation delays for devices in the *decode* phase, it conversely results in an increased TTFT for devices in the *prefill* phase. This dichotomy underscores the necessity of optimizing chunk size to strike an optimal balance for batched inference in the cloud

2.4 Our Proposed Framework

While U-shaped inference enhances the privacy protection of raw tokens, it inevitably introduces significant communication overhead due to the frequent transmission of hidden states. On the other hand, although speculative decoding reduces the device-cloud interaction frequency and accelerates inference speed, it presents inherent privacy vulnerabilities. Considering the complementary strengths of U-shaped inference and speculative decoding, we propose to integrate U-shaped inference with speculative decoding and develop a new and novel device-cloud collaborative inference framework named HAT. Figure 2 illustrates the HAT framework, where an LLM with 32 decoder layers is exemplified. In HAT, the shallow layers (*e.g.*, layers #0 and #1) and the head of LLM are deployed on end devices as input and output submodels to enhance privacy, while the middle submodel, encompassing other 30 decoder layers, resides in the cloud. Besides, a lightweight adapter network distilled from the middle submodel is deployed on devices to form the draft model, which performs speculative decoding to accelerate inference and maintain promising accuracy. As shown in Figure 2, the main inference process of HAT involves local drafting (denoted in dashed green line) and device-cloud verification (denoted in solid red line), which looks like a "top hat".

According to the analysis in Sec. 2.3, chunking the long prompts into multiple short prompts can alleviate in-cloud computation delays for devices in *decode* phase. Upon the multiple short prompts, HAT can maximize the overlap between device-side transmission of chunked hidden states and in-cloud computation of previously adjacent chunked hidden states to reduce the TTFT for devices in *prefill* phase (more details in Sec. 3.3). Besides, considering the characteristic of speculative decoding, HAT allows devices to continue local drafting (*i.e.*, parallel drafting) while awaiting verification results from the LLM in *decode* phase so as to further utilize the computational resources of devices and improve inference speed (more details in Sec. 3.5).

3 System Design

3.1 System Overview

The system design of HAT is illustrated in Figure 3. HAT consists of four key modules, *i.e.*, *state monitoring*, *prompt chunking*, *speculative decoding*, and *parallel drafting*. Through batched inference, the cloud processes requests from multiple devices in *prefill* phase and *decode* phase simultaneously. At regular intervals, the state monitoring module gathers critical data on the cloud's workload and the state information of all devices, including their computing and communication capabilities. When a request arrives at a device, the prompt chunking module determines an optimal chunk size tailored to the device's capabilities to complete *prefill* phase, in which the device divides the prompt into specified chunks and processes them in a pipelined fashion. Following the *prefill* phase, the speculative decoding module orchestrates an iterative process where the device drafts potential continuations and verifies them against the cloud's predictions in *decode* phase. Concurrently, the parallel drafting module facilitates the device in generating a draft sequence for the subsequent round of speculative decoding during the verification stage.

3.2 State Monitoring Module

To determine the appropriate chunk size, it is necessary to continuously collect multiple rounds of state information of the cloud (*e.g.*, workload) and all devices (*e.g.*, computing and communication capabilities).

Concretely, the cloud serves requests from different devices at different moments, resulting in a dynamic workload. We denote the batched token size (*i.e.*, the number of processed tokens in a batch) and the in-cloud computation delay for the batch in round t as μ^t and η^t, respectively. As proxy metrics, we adopt μ^t and η^t, which can be recorded by the cloud directly, to indicate the workload of the cloud in round t. Besides, the cloud maintains a predictive function $g^t(\cdot)$ for predicting the in-cloud computation delays associated with different chunk sizes. During the information collection of round t, the cloud collects the latest $\hat{\mu}^t$ and $\hat{\eta}^t$, and updates the predictive function $g^t(\cdot)$. Moreover, to improve the robustness of the estimation, we introduce moving averages with historical workload [28]. Accordingly,

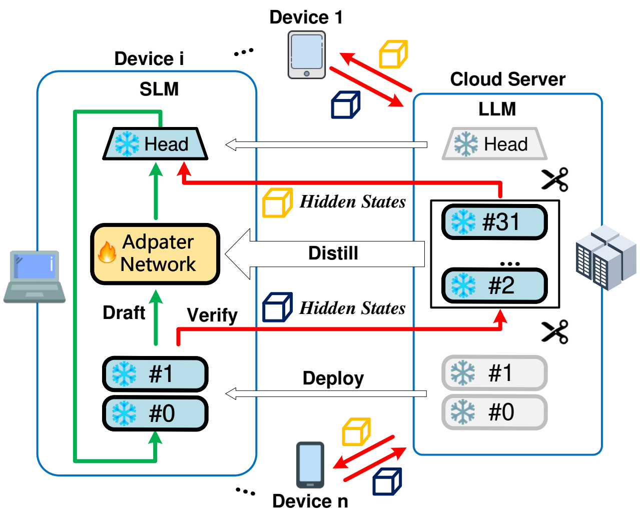


图 2：HAT 示意图。

云计算延迟了*decode*阶段设备的云计算延迟，相反导致*prefill*阶段设备的TTFT增加。这种二分法强调了优化块大小的必要性，以实现云中批量推理的最佳平衡

2.4 我们提出的框架

虽然U形推理增强了原始令牌的隐私保护，但由于隐藏状态的频繁传输，它不可避免地引入了显着的通信开销。另一方面，推测性解码虽然降低了端云交互频率，加快了推理速度，但也带来了固有的隐私漏洞。考虑到U型推理和推测解码的优势互补，我们建议将U型推理与推测解码相结合，开发一种新颖的端云协同推理框架，名为HAT。图 2 说明了 HAT 框架，其中以具有 32 个解码器层的 LLM 为例。在 HAT 中，浅层（*e.g.*、#0 和 #1 层）和 LLM 的头部作为输入和输出子模型部署在终端设备上，以增强隐私性，而包含其他 30 个解码器层的中间子模型则驻留在云端。此外，从中间子模型中提取的轻量级适配器网络被部署在设备上以形成草稿模型，该模型执行推测解码以加速推理并保持有希望的准确性。如图2所示，HAT的主要推理过程包括本地起草（绿色虚线表示）和端云验证（红色实线表示），看起来像一顶“大礼帽”。

根据第二节的分析。 2.3，将长提示分成多个短提示可以减轻*decode*阶段设备的云内计算延迟。根据多个短提示，HAT 可以最大化分块隐藏状态的设备端传输与先前相邻分块隐藏状态的云计算之间的重叠，以减少 *prefill* 阶段设备的 TTFT（更多详细信息请参见第 3.3 节）。此外，考虑到推测解码的特性，HAT允许设备在等待的同时继续本地起草（*i.e.*，并行起草）

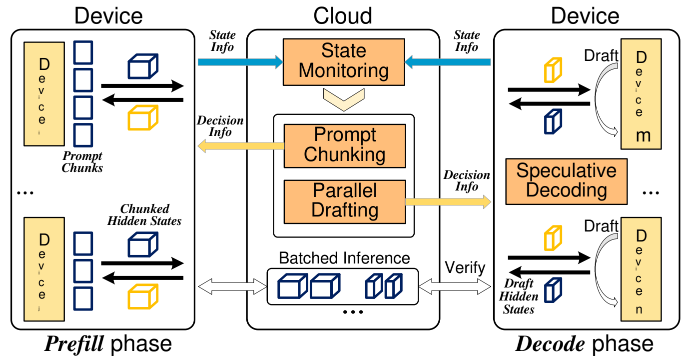


图 3：HAT 的系统概览。

在*decode*阶段验证LLM的结果，以进一步利用设备的计算资源并提高推理速度（更多详细信息请参见第3.5节）。

3 系统设计

3.1 系统概述

HAT的系统设计如图3所示。HAT由四个关键模块组成：*i.e.*、*state monitoring*、*prompt*、*chunking*、*speculative decoding*和*parallel drafting*。通过批量推理，云在*prefill*阶段和*decode*阶段同时处理来自多个设备的请求。状态监控模块定期收集有关云工作负载的关键数据以及所有设备的状态信息，包括其计算和通信能力。当请求到达设备时，提示分块模块会确定适合设备能力的最佳块大小，以完成 *prefill* 阶段，在该阶段，设备将提示划分为指定的块并以管道方式处理它们。在 *prefill* 阶段之后，推测性解码模块会编排一个迭代过程，其中设备起草潜在的延续，并在 *decode* 阶段根据云的预测对其进行验证。同时，并行起草模块有助于设备在验证阶段生成用于下一轮推测解码的草案序列。

3.2 状态监控模块

为了确定合适的块大小，需要持续收集云（*e.g.*，工作负载）和所有设备（*e.g.*，计算和通信能力）的多轮状态信息。

具体来说，云在不同时刻服务来自不同设备的请求，从而产生动态工作负载。我们将第 t 轮中批次的令牌大小（*i.e.*，批次中已处理的令牌数量）和批次的云内计算延迟分别表示为 μ^t 和 η^t。作为代理指标，我们采用云可以直接记录的μ^t和η^t来指示云在第t轮的工作负载。此外，云还维护一个预测函数$g^t(\cdot)$，用于预测与不同块大小相关的云内计算延迟。在第t轮信息采集过程中，云端采集最新的$\hat{\mu}^t$和$\hat{\eta}^t$，并更新预测函数$g^t(\cdot)$。此外，为了提高估计的稳健性，我们引入了具有历史工作量的移动平均线[28]。因此，

the cloud estimates μ^t and $g^t(\cdot)$ in round t by calculating the moving average with $\alpha \in [0, 1]$ (*e.g.*, 0.8 in our experiments) as:

$$\mu^t = \alpha \cdot \mu^{t-1} + (1 - \alpha) \cdot \hat{\mu}^t \tag{1}$$

$$g^t(\mu^t) = \alpha \cdot g^{t-1}(\mu^t) + (1 - \alpha) \cdot \hat{\eta}^t \tag{2}$$

With this predictive function, the cloud can predict the computation delay based on batched token size, enabling more efficient chunk size optimization.

Additionally, the collection of time-varying computing and communication capabilities of devices is necessary for chunk size optimization. We use γ_i^t, $\beta_{i,up}^t$, and $\beta_{i,down}^t$ to denote the drafting delay, uploading, and downloading bandwidths of device i in round t, respectively, which can be recorded and computed by devices directly. Besides, we also compute their moving average based on their historical information. Since the size of state information is much smaller than that of hidden states, it is reasonable to ignore the computation/communication overhead for state monitoring [29].

3.3 Prompt Chunking Module

Upon the collected state information, the cloud determines diverse and appropriate chunk sizes for the devices to complete prefilling with prompt chunking. Figure 4 illustrates the differences in processing the prompts of a device with/without the prompt chunking mechanism in *prefill* phase. Concretely, this mechanism enables the device to divide its long prompt into multiple shorter chunks with a certain chunk size, and process these chunks in a pipelined manner. By parallelizing the computation and transmission of hidden states, the uploading of chunked hidden states can overlap with other operations like in-cloud computation and downloading, thereby reducing the delay in *prefill* phase. Moreover, processing chunked hidden states in the cloud demands fewer computational resources, which incurs less in-cloud computation delay for other devices in *decode* phase.

Given a long prompt, the chunk size critically influences both communication and in-cloud computation delays, and the total number of chunks. As indicated in Sec. 2.3, although communication delay exhibits an approximately linear relationship with chunk size, the in-cloud computation delay demonstrates limited reduction with decreasing chunk size, particularly at smaller scales. This phenomenon can result in accumulated computation delays across all chunks that exceed the communication delay within the pipeline framework. Conversely, larger chunk sizes may introduce substantial per-chunk delays, thereby reducing parallelization efficiency. Therefore, optimal chunk size selection should balance granularity to minimize the in-cloud computation delay for each chunk, while maintaining sufficient communication delay to overlap with preceding chunk processing.

Furthermore, each incoming chunk experiences a delay in waiting for the current batch to be processed in the cloud, making the total in-cloud delay the sum of waiting and in-cloud computation delays. The maximum waiting delay approximates the average in-cloud computation delay $g^t(\mu^t)$, while the computation delay for a chunk of size X_i

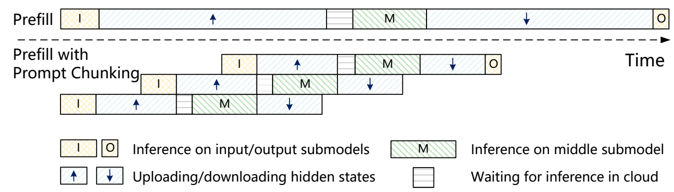


Fig. 4: Illustration of Prompt Chunking.

from device i is estimated as $g^t(\mu^t + X_i)$. In practice, the cloud usually utilizes multiple GPUs for pipeline-parallel inference, where computation delay per GPU is inversely proportional to the number of GPUs (denoted as pipeline length). This architecture eliminates the need to wait for the previous inference to be finished across the entire model in the cloud. Accordingly, the optimal chunk size X_i is calculated with pipeline length P by the following formula:

$$\frac{X_i \cdot A}{\beta_{i,up}^t} = \frac{g^t(\mu^t) + g^t(\mu^t + X_i)}{P} \tag{3}$$

where A is the size of the hidden state per token. We will further explore the impact of pipeline length scale on inference delay of HAT in Sec. 4.5.

3.4 Speculative Decoding Module

We first introduce the construction and training of the lightweight adapter network. Generally, given an LLM w_L^n, which consists of an output head and n decoder layers. The lightweight adapter network Λ is constructed and combined with the LLM's output head H_L and first m decoder layers w_L^m (m less than 10% of n) to form a draft model $w_S = H_L \circ \Lambda \circ w_L^m$. Λ has the same structure as the self-attention module of the decoder layer, as the self-attention module has fewer parameters and computation delay than the feed-forward network. To enable efficient drafting, Λ is trained by knowledge distillation to learn the middle submodel of w_L^n. Following [24], we use Smooth L1 loss (L_{sl}) and Cross-Entropy loss (L_{ce}) as the training loss function for knowledge distillation:

$$Loss = L_{sl}(f_{i+1}^L, f_{i+1}^S) + w_{ce} L_{ce}(H_L(f_{i+1}^L), H_L(f_{i+1}^S)) \tag{4}$$

where f_{i+1}^L and f_{i+1}^S denote the hidden states of the new token t_{i+1} in w_L^n and w_S, respectively, before they are fed into the output head. The weight w_{ce} controls the contribution of Cross-Entropy loss and is typically set to 0.1.

The draft model w_S and LLM w_L^n collaborate in performing multiple rounds of speculative decoding. Specifically, each round of speculative decoding begins with the drafting stage, where w_S on the device performs multiple autoregressive inferences to generate a draft sequence. During the subsequent verification stage, the draft sequence is first processed by the shallow layers w_L^m to produce shallow hidden states, which are then transmitted to the cloud for further inference by the middle submodel of w_L^n to produce deep hidden states. The deep hidden states are then sent back to the device, where the final inference result is generated by the LLM's head H_L. The draft tokens with the same inference result of LLM will be accepted, and a new token will be generated from the last accepted draft token. Subsequently, this new token is used as input for the next round of speculative decoding.

云通过使用 $\alpha \in [0, 1]$（*e.g.*，在我们的实验中为 0.8）计算移动平均值来估计第 t 轮中的 μ^t 和 $g^t(\cdot)$：

$$\mu^t = \alpha \cdot \mu^{t-1} + (1-\alpha) \cdot \hat{\mu}^t \quad (1)$$

$$g^t(\mu^t) = \alpha \cdot g^{t-1}(\mu^t) + (1-\alpha) \cdot \hat{\eta}^t \quad (2)$$

通过此预测功能，云可以根据批量令牌大小预测计算延迟，从而实现更有效的块大小优化。

此外，收集设备的时变计算和通信能力对于块大小优化是必要的。我们用γ_i^t、$\beta_{i,up}^t$和$\beta_{i,down}^t$分别表示第t轮中设备i的起草延迟、上传和下载带宽，可以由设备直接记录和计算。此外，我们还根据他们的历史信息计算他们的移动平均线。由于状态信息的大小远小于隐藏状态的大小，因此忽略状态监视的计算/通信开销是合理的[29]。

3.3 提示分块模块

根据收集到的状态信息，云为设备确定不同且适当的块大小，以通过快速分块完成预填充。图 4 说明了在 *prefill* 阶段有/没有提示分块机制的设备处理提示的差异。具体来说，这种机制使得设备能够将其长提示符分成多个具有一定块大小的较短块，并以流水线方式处理这些块。通过并行隐藏状态的计算和传输，分块隐藏状态的上传可以与云内计算和下载等其他操作重叠，从而减少*prefill*阶段的延迟。此外，在云中处理分块隐藏状态需要更少的计算资源，这会在*decode*阶段为其他设备带来更少的云内计算延迟。

给定较长的提示，块大小会严重影响通信和云内计算延迟以及块的总数。如第 2 节所示。 2.3，尽管通信延迟与块大小呈现近似线性关系，但云内计算延迟随着块大小的减小而减少有限，特别是在较小的规模下。这种现象可能会导致所有块的累积计算延迟超过管道框架内的通信延迟。相反，较大的块大小可能会引入大量的每块延迟，从而降低并行化效率。因此，最佳的块大小选择应该平衡粒度，以最小化每个块的云内计算延迟，同时保持足够的通信延迟以与前面的块处理重叠。

此外，每个传入块在等待当前批次在云中处理时都会经历延迟，从而使云中总延迟成为等待延迟和云中计算延迟的总和。最大等待延迟近似于平均云计算延迟 $g^t(\mu^t)$，而大小为 X_i 的块的计算延迟

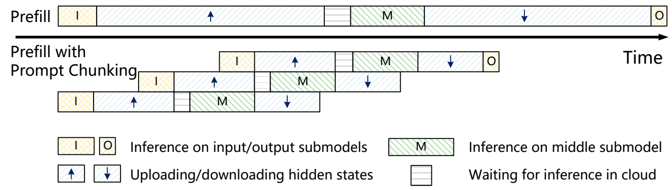


图 4：提示分块的图示。

来自设备 i 的估计为 $g^t(\mu^t + X_i)$。在实践中，云通常利用多个 GPU 进行管道并行推理，其中每个 GPU 的计算延迟与 GPU 数量（表示为管道长度）成反比。这种架构无需等待云中整个模型完成先前的推理。因此，最佳块大小 X_i 与管道长度 P 通过以下公式计算：

$$\frac{X_i \cdot A}{\beta_{i,up}^t} = \frac{g^t(\mu^t) + g^t(\mu^t + X_i)}{P} \quad (3)$$

其中 A 是每个标记的隐藏状态的大小。我们将在第 2 节中进一步探讨管道长度规模对 HAT 推理延迟的影响。 4.5.

3.4 推测解码模块

我们首先介绍轻量级适配器网络的构建和训练。一般来说，给定一个 LLM w_L^n，它由输出头和 n 解码器层组成。构建轻量级适配器网络 Λ，并将其与 LLM 的输出头 H_L 和小于 n) 10% 的第一个 m 解码器层 w_L^m (m 相结合，形成草案模型 $w_S = H_L \circ \Lambda \circ w_L^m$。 Λ与解码器层的自注意力模块具有相同的结构，因为自注意力模块比前馈网络具有更少的参数和计算延迟。为了实现高效的绘图，通过知识蒸馏来训练Λ以学习w_L^n的中间子模型。按照[24]，我们使用 Smooth L1 loss (L_{sl}) 和 Cross-Entropy loss (L_{ce}) 作为知识蒸馏的训练损失函数：

$$Loss = L_{sl}(f_{i+1}^L, f_{i+1}^S) + w_{ce} L_{ce}(H_L(f_{i+1}^L), H_L(f_{i+1}^S)) \quad (4)$$

其中f_{i+1}^L和f_{i+1}^S分别表示新标记t_{i+1}在w_L^n和w_S中的隐藏状态，然后它们被送入输出头。权重 w_{ce} 控制交叉熵损失的贡献，通常设置为 0.1。

草案模型 w_S 和 LLM w_L^n 协作执行多轮推测解码。具体来说，每轮推测解码都从起草阶段开始，其中设备上的 w_S 执行多个自回归推理以生成草案序列。在随后的验证阶段，草稿序列首先由浅层w_L^m处理以产生浅隐藏状态，然后将其传输到云端以由w_L^n的中间子模型进一步推理以产生深层隐藏状态。然后深层隐藏状态被发送回设备，最终的推理结果由 LLM 的头 H_L 生成。与LLM推理结果相同的草稿代币将被接受，并且将从最后接受的草稿代币生成新的代币。随后，这个新令牌被用作下一轮推测解码的输入。

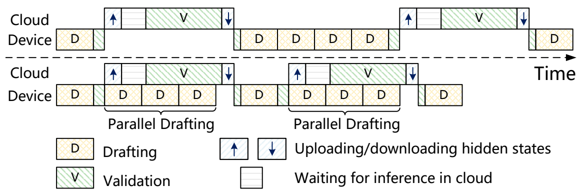


Fig. 5: Illustration of Parallel Drafting.

During the drafting stage, the efficiency of speculative decoding is greatly influenced by the drafting sequence length n, which reflects the number of drafting steps. Concretely, if the draft sequence length n is too short, some simpler tokens that would have been accepted may be missed. Conversely, a long draft sequence length can lead to the squandering of computational resources on challenging tokens, increasing drafting overhead. To optimize the drafting process, we implement a drafting threshold $\eta \in [0, 1]$ (*e.g.*, 0.6 in our experiments) to adaptively stop the draft when the softmax probability of the draft token falls below the threshold :

$$\max_{n} w_S(t_{i+n}) < \eta \tag{5}$$

where $w_S(t_{i+n})$ denoted the softmax probability of the n-th token generated by w_S during the drafting stage.

3.5 Parallel Drafting Module

Due to the limited computing capabilities of devices, the local computation for drafting incurs non-negligible delays, reducing the efficiency of speculative decoding. Therefore, we introduce the parallel drafting mechanism to parallel the drafting and verification stages, as shown in Figure 5. This mechanism enables devices to generate the draft sequence for the next round of speculative decoding in advance, fully utilizing idle computational resources of devices during the verification stage.

Specifically, the device takes the top-k tokens generated from the last inference step in the drafting stage as candidate tokens. During the verification stage, these candidate tokens serve as the initial input to generate candidate draft sequences in parallel. According to Eq. (5), the softmax probability of the last draft token falls below the drafting threshold, resulting in a relatively low probability of being accepted. Therefore, the last draft token may be corrected into a new token in the verification stage. If this new token matches one of the top-k candidate tokens, the corresponding candidate draft sequence will be used for the subsequent round of speculative decoding.

Based on the collected state information, the cloud selects an appropriate number of inference steps to guide the devices to generate candidate draft sequences during the verification stage. To fully utilize the idle computational resources of devices without interference with the verification stage, the generation process should be completed before receiving the inference result from the cloud. Therefore, we consider the case where the in-cloud delay of the verification stage is minimized (*i.e.*, no wait time in the cloud). We use λ_i to represent the number of inference steps for parallel drafting in device i, which is calculated as follows:

$$\lambda_i = \left\lfloor \frac{\frac{\mu_i \cdot A}{\beta_{i,up}^t} + g^t(\mu^t) + \frac{\mu_i \cdot A}{\beta_{i,down}^t}}{\gamma_i^t} \right\rfloor \tag{6}$$

TABLE 2: Device Technical Specifications.

	AI Performance	GPU Type
AGX Xavier	32 TOPS	512-core Volta
AGX Orin	200 TOPS	1792-core Ampere
	CPU Type	**ROM**
AGX Xavier	8-core Carmel ARM 8	32 GB LPDDR4x
AGX Orin	8-core Cortex ARM 8	32 GB LPDDR5

TABLE 3: Prompt Token Length and Models.

Dataset	Avg	P90	Std	Model
SpecBench	351.2	891.0	397.3	Vicuna-7B
CNN/DM	1036.6	1772.0	511.8	Vicuna-13B

where μ_i is the draft sequence length of device i in the current round of speculative decoding.

4 Experiments and Evaluation

4.1 Experimental Settings

System Deployment. We conduct extensive experiments to evaluate the performance of HAT on an end-to-end hardware prototype system. Specifically, we employ a deep learning workstation as the cloud server, which is equipped with an Intel(R) Xeon(R) Platinum 8358P CPU, 8 NVIDIA GeForce RTX A6000 GPUs, and 512 GB RAM. In addition, we specify 30 NVIDIA Jetson kits, including 20 Jetson AGX Xavier and 10 Jetson AGX Orin as devices to construct a heterogeneous system. The detailed technical specifications of Jetson AGX Xavier and Orin are listed in Table 2.

In the experiments, we build the software platform based on Docker Swarm [30], [31] and the PyTorch deep learning library [32]. The Docker Swarm, a distributed software development kit, facilitates the construction of a distributed system and enables the monitoring of each device's operational status. The PyTorch library facilitates the implementation of model inference across devices. Additionally, MPI (Message Passing Interface) [33] which includes a collection of sending and receiving functions, is implemented to streamline communication between devices and the server. Furthermore, we incorporate FlashAttention [34] and FlashInfer [35] kernels to support page attention and continuous batching, and we use NCCL [36] for pipeline communication between GPUs.

Setting of System Heterogeneity. To enable the devices with heterogeneous computing and communication capabilities, we present the following experimental settings.

1) ***For Computation.*** All the Jetson devices can be configured to work with different modes, specifying the number of working CPUs and the frequency of CPU/GPU, so that they can work with different computing capabilities. For instance, the AGX Orin with the highest performance mode (*i.e.*, mode 0) can achieve inference by 10× faster than the AGX Xavier with the lowest performance mode (*i.e.*, mode 1). To simulate the time-varying resources of devices, we randomly change the modes of devices after every 5 requests generated.

2) ***For Communication.*** All devices are connected to the server via WiFi routers. We group devices into three groups, each containing 10 devices. These groups are then placed at different locations, *i.e.*, 2m, 8m, and 14m away from the WiFi routers. Due to random channel noise and competition

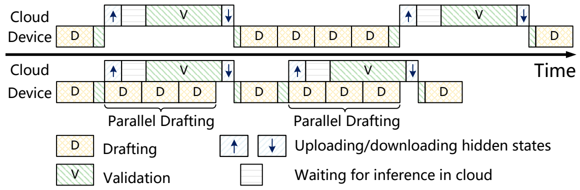


图 5：平行绘图示意图。

在起草阶段，推测解码的效率很大程度上受到起草序列长度n的影响，它反映了起草步骤的数量。具体来说，如果草案序列长度n太短，则可能会丢失一些本来可以接受的更简单的标记。相反，较长的草稿序列长度可能会导致在具有挑战性的标记上浪费计算资源，从而增加草稿开销。为了优化起草过程，我们实现了起草阈值$\eta \in [0, 1]$（实验中为*e.g.*，0.6），以在草稿令牌的softmax概率低于阈值时自适应地停止草稿：

$$\max_{n} w_S(t_{i+n}) < \eta \tag{5}$$

其中 $w_S(t_{i+n})$ 表示在起草阶段由 w_S 生成的第 n 个令牌的 softmax 概率。

3.5 并行绘图模块

由于设备计算能力有限，绘图的本地计算会产生不可忽略的延迟，降低了推测解码的效率。因此，我们引入并行起草机制来并行起草和验证阶段，如图5所示。该机制使设备能够提前生成下一轮推测解码的草案序列，充分利用设备在验证阶段的空闲计算资源。

具体来说，设备将起草阶段最后一个推理步骤生成的 top-k 令牌作为候选令牌。在验证阶段，这些候选标记用作并行生成候选草稿序列的初始输入。根据方程。(5)，最后一个草稿令牌的softmax概率低于草稿阈值，导致被接受的概率相对较低。因此，最后的草稿令牌可以在验证阶段被修正为新的令牌。如果这个新令牌与前 k 个候选令牌之一匹配，则相应的候选草稿序列将用于下一轮推测解码。

根据收集的状态信息，云端选择适当数量的推理步骤来指导设备在验证阶段生成候选草稿序列。为了充分利用设备的空闲计算资源而不干扰验证阶段，生成过程应在从云端接收推理结果之前完成。因此，我们考虑验证阶段的云端延迟最小化的情况（*i.e.*，云端无等待时间）。我们用λ_i来表示设备i中并行绘图的推理步数，其计算如下：

$$\lambda_i = \left\lfloor \frac{\frac{\mu_i \cdot A}{\beta^t_{i,up}} + g^t(\mu^t) + \frac{\mu_i \cdot A}{\beta^t_{i,down}}}{\gamma^t_i} \right\rfloor \tag{6}$$

其中μ_i是当前轮推测解码中设备i的草案序列长度。

表 2：器件技术规格。

	AI Performance	GPU Type
AGX Xavier	32 TOPS	512-core Volta
AGX Orin	200 TOPS	1792-core Ampere
	CPU Type	**ROM**
AGX Xavier	8-core Carmel ARM 8	32 GB LPDDR4x
AGX Orin	8-core Cortex ARM 8	32 GB LPDDR5

表 3：提示令牌长度和模型。

Dataset	Avg	P90	Std	Model
SpecBench	351.2	891.0	397.3	Vicuna-7B
CNN/DM	1036.6	1772.0	511.8	Vicuna-13B

4 实验与评价

4.1 实验设置

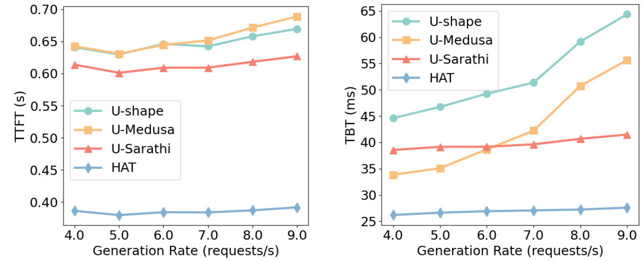
系统部署。我们进行了大量的实验来评估 HAT 在端到端硬件原型系统上的性能。具体来说，我们采用深度学习工作站作为云服务器，它配备了 Intel(R) Xeon(R) Platinum 8358P CPU、8 个 NVIDIA GeForce RTX A6000 GPU 和 512 GB RAM。此外，我们指定了 30 个 NVIDIA Jetson 套件，其中包括 20 个 Jetson AGX Xavier 和 10 个 Jetson AGX Orin 作为构建异构系统的设备。 Jetson AGX Xavier和Orin的详细技术规格如表2所示。

在实验中，我们基于Docker Swarm [30]、[31]和PyTorch深度学习库[32]构建了软件平台。 Docker Swarm是一个分布式软件开发工具包，可以方便地构建分布式系统并监控每个设备的运行状态。 PyTorch 库有助于跨设备实现模型推理。此外，还实现了 MPI（消息传递接口）[33]，其中包括发送和接收功能的集合，以简化设备和服务器之间的通信。此外，我们结合了 FlashAttention [34] 和 FlashInfer [35] 内核来支持页面注意力和连续批处理，并且我们使用 NCCL [36] 进行 GPU 之间的管道通信。

系统异构性的设置。为了使设备具有异构计算和通信能力，我们提出了以下实验设置。

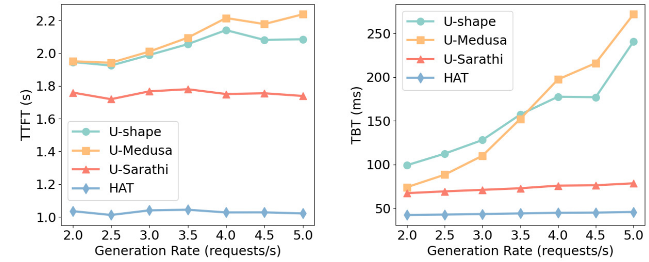
1）***For Computation.***所有Jetson设备都可以配置为工作在不同的模式下，指定工作CPU的数量和CPU/GPU的频率，以便它们可以工作在不同的计算能力上。例如，具有最高性能模式（*i.e.*，模式 0）的 AGX Orin 的推理速度比具有最低性能模式（*i.e.*，模式 1）的 AGX Xavier 快 10×。为了模拟设备随时间变化的资源，我们在每生成 5 个请求后随机更改设备的模式。

2) ***For Communication.*** 所有设备都通过 WiFi 路由器连接到服务器。我们将设备分为三组，每组包含 10 个设备。然后将这些组放置在距离 WiFi 路由器 2m、8m 和 14m 的不同位置（*i.e.*）。由于随机信道噪声和竞争



(a) TTFT (b) TBT

Fig. 6: Delay with Different Request Generation Rates on SpecBench.



(a) TTFT (b) TBT

Fig. 7: Delay with Different Request Generation Rates on CNN/DM.

among devices, the bandwidth between the server and devices dynamically varies. Devices' bandwidth ranges from 5MB/s to 10MB/s for uplinks and from 10MB/s to 15MB/s for downlinks, as measured by iperf3 [37].

Datasets and Models. We evaluate HAT's performance on two datasets with Vicuna model fine-tuned from LLama.

1) ***SpecBench*** [27] is a versatile dataset encompassing a range of tasks such as translation, summarization, mathematical reasoning, *etc.*. The tasks in SpecBench exhibit significant variations in prompt length. For instance, the prompts for summarisation tasks have an average length of 877.3, while the average prompt length for the translation tasks is only 82.5. For this dataset, we adopt a famous Vicuna-7B LLM, which includes 32 decoder layers with 32 attention heads, and its hidden size is 4096.

2) ***CNN/DM*** [38] focuses on text summarization and comprises 11490 news articles, each associated with long prompts. Concretely, the tasks of CNN/DM have prompts with an average length of 1036.6. The specific prompt token lengths for each dataset are listed in Table 3. For this dataset, we utilize a more robust Vicuna-13B LLM, which includes 40 decoder layers with 40 attention heads, and its hidden size is 5120.

Baselines. We measure the effectiveness of HAT through a comparison with three baselines.

1) ***U-shape*** [16], [39]**.** It is a famous split inference approach that deploys LLM's head (output submodel) and shallow layers (input submodel) to devices to enhance privacy, while the computationally intensive middle submodel resides in the server. Note that given our primary focus on privacy-preserving inference, all baselines are adapted within the U-shaped framework. We specifically emphasize the comparative analysis of inference efficiency across various modified frameworks.

2) ***U-Medusa.*** We integrate Medusa [25], an efficient speculative decoding approach, within the U-shaped inference framework as U-Medusa. Medusa constructs and trains 4 Medusa heads to predict the next 4 tokens per inference step. Therefore, U-Medusa deploys 4 Medusa heads with LLM's input and output submodels to devices to enhance privacy. Additionally, it employs a tree verification method that aggregates candidate tokens from Medusa heads to extend the number of accepted draft tokens.

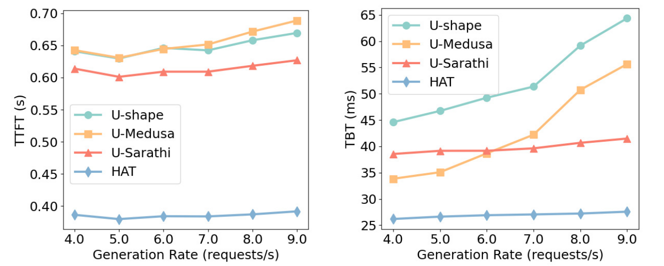
3) ***U-Sarathi.*** We integrate Sarathi-Serve [21], a state-of-the-art LLM serving system, within the U-shaped inference framework as U-Sarathi. The server in U-Sarathi chunks prompts to fully utilize its computational resources, minimizing the bubbles in pipeline-parallel inference and reducing the computation delay for devices in *decode* phase.

Metrics. There are two primary delay metrics for LLM inference performance: time-to-first-token (TTFT) and time-between-tokens (TBT). TTFT measures the delay of generating the first output token from the moment a request arrives in *prefill* phase, which reflects the initial responsiveness of the system. Besides, TBT measures the interval between the generation of consecutive output tokens during *decode* phase, which affects the overall perceived fluidity of the response. Moreover, LLM inference services must meet stringent delay requirements to maintain the quality of service. Accordingly, we establish two service level agreements (SLAs) for *prefill* and *decode* to assess the SLA compliance rate, *i.e.*, the percentage of requests that satisfy these SLA targets.

Experimental Parameters. We deploy the first 2 shallow layers of the Vicuna-7B, and the first 3 shallow layers of the Vicuna-13B, along with their respective heads, to devices. In addition, considering the trade-off between the communication delay of hidden states and accept length, we choose tree verification of size 8 for U-Medusa to enhance the efficiency of speculative decoding. For U-Sarathi, we choose chunk sizes of 128 and 256 for SpecBench and CNN/DM datasets, respectively. Moreover, both the U-Medusa heads and HAT's constructed network Λ are trained on the ShareGPT dataset [40]. Furthermore, the maximum generation lengths for all datasets are set to 128 tokens.

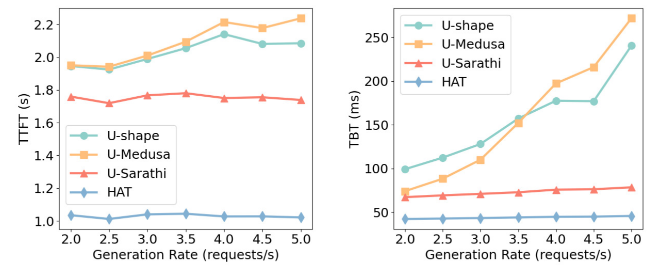
4.2 Overall Performance

Firstly, we conduct a set of experiments on two datasets with different request generation rates (*i.e.*, the number of requests per second) to evaluate the performance of HAT and baselines. In the experiments, devices generate requests following a Poisson process, and the server employs 4 GPUs for pipeline-parallel inference (*i.e.*, the server's pipeline length is 4). As illustrated in Figures 6-7, HAT consistently achieves the lowest TTFT and TBT among all approaches. For instance, Figure 6(a) shows that at a generation rate of 6 requests/s on SpecBench, HAT achieves the TTFT of 384.2ms, significantly lower than the TTFT of U-Sarathi (609.2ms), U-Medusa (644.6ms), and U-shape (646.2ms). Moreover, by Figure 6(b), when the generation rate is 4 requests/s, HAT reduces the TBT by 31.9%, 22.5% and 41.3%, compared to U-Sarathi, U-Medusa and U-shape, respectively. Besides, as the generation rate increases from 4 to



(a) TTFT (b) TBT

图 6：SpecBench 上不同请求生成率的延迟。



(a) TTFT (b) TBT

图 7：CNN/DM 上不同请求生成率的延迟。

在设备之间，服务器和设备之间的带宽动态变化。根据 iperf 3 [37] 的测量，设备的上行链路带宽范围为 5MB/s 至 10MB/s ，下行链路带宽范围为 10MB/s 至 15MB/s。

数据集和模型。我们使用从 LLama 微调的 Vicuna 模型来评估 HAT 在两个数据集上的性能。

1）***SpecBench*** [27] 是一个多功能数据集，涵盖一系列任务，例如翻译、摘要、数学推理、*etc*。 SpecBench 中的任务在提示长度上表现出显着差异。例如，摘要任务的提示平均长度为 877.3，而翻译任务的平均提示长度仅为 82.5。对于这个数据集，我们采用了著名的Vicuna-7B LLM，它包括32个解码器层和32个注意力头，其隐藏大小为4096。

2) ***CNN/DM*** [38] 专注于文本摘要，包含 11490 篇新闻文章，每一篇都与长提示相关。具体来说，CNN/DM的任务有平均长度为1036.6的提示。表3列出了每个数据集的具体提示标记长度。对于该数据集，我们使用更强大的Vicuna-13B LLM，其中包括40个解码器层和40个注意力头，其隐藏大小为5120。

基线。我们通过与三个基线的比较来衡量 HAT 的有效性。

1）***U-shape*** [16]，[39]。这是一种著名的分割推理方法，它将LLM的头部（输出子模型）和浅层（输入子模型）部署到设备上以增强隐私性，而计算密集型的中间子模型则驻留在服务器中。请注意，鉴于我们主要关注隐私保护推理，所有基线均在 U 形框架内进行调整。我们特别强调对各种修改框架的推理效率的比较分析。

2）***U-Medusa***.我们将 Medusa [25]（一种有效的推测解码方法）集成到 U 形推理框架中，称为 U-Medusa。 Medusa 构建并训练 4 个 Medusa 头来预测每个推理步骤的接下来 4 个标记。因此，U-Medusa在设备上部署了4个带有LLM输入和输出子模型的Medusa头，以增强隐私性。此外，它还采用树验证方法，聚合来自 Medusa 头的候选令牌，以扩展接受的草案令牌的数量。

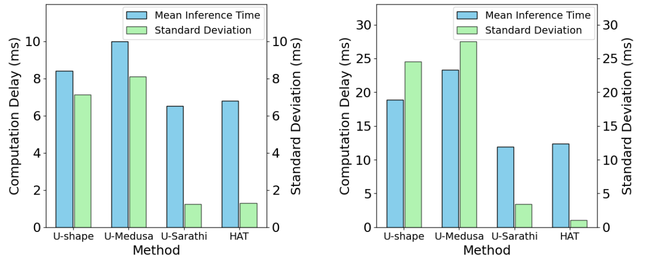
3) ***U-Sarathi***. 我们将最先进的 LLM 服务系统 Sarathi-Serve [21] 集成到 U 形推理框架中，称为 U-Sarathi。 U-Sarathi 块中的服务器提示充分利用其计算资源，最大限度地减少管道并行推理中的气泡，并减少*decode*阶段设备的计算延迟。

指标。 LLM 推理性能有两个主要延迟指标：首次令牌时间 (TTFT) 和令牌之间时间 (TBT)。 TTFT测量从请求到达*prefill*阶段起生成第一个输出令牌的延迟，这反映了系统的初始响应能力。此外，TBT 测量 *decode* 阶段连续输出令牌生成之间的间隔，这会影响响应的整体感知流动性。此外，LLM 推理服务必须满足严格的延迟要求，以维持服务质量。因此，我们为 *prefill* 和 *decode* 建立了两个服务级别协议 (SLA)，以评估 SLA 合规率 *i.e.*，即满足这些 SLA 目标的请求的百分比。

实验参数。我们将 Vicuna-7B 的前 2 个浅层和 Vicuna-13B 的前 3 个浅层及其各自的头部部署到设备上。此外，考虑到隐藏状态的通信延迟和接受长度之间的权衡，我们为U-Medusa选择大小为8的树验证，以提高推测解码的效率。对于 U-Sarathi，我们分别为 SpecBench 和 CNN/DM 数据集选择 128 和 256 的块大小。此外，U-Medusa 头和 HAT 构建的网络 Λ 都是在 ShareGPT 数据集上训练的 [40]。此外，所有数据集的最大生成长度设置为 128 个标记。

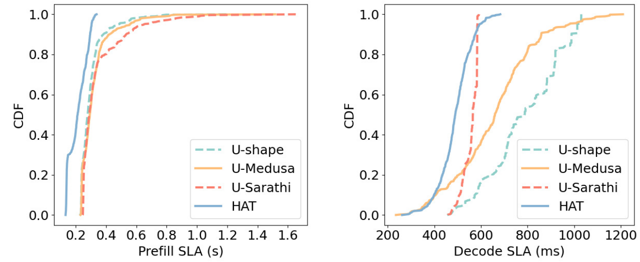
4.2 整体性能

首先，我们在两个具有不同请求生成率（*i.e.*，每秒请求数）的数据集上进行一组实验，以评估 HAT 和基线的性能。实验中，设备按照泊松过程生成请求，服务器使用 4 个 GPU 进行管道并行推理（*i.e.*，服务器管道长度为 4）。如图 6-7 所示，HAT 在所有方法中始终实现最低的 TTFT 和 TBT。例如，图6（a）显示，在SpecBench上6个请求/秒的生成速率下，HAT实现了384.2ms的TTFT，显着低于U-Sarathi（609.2ms）、U-Medusa（644.6ms）和U-shape（646.2ms）的TTFT。此外，由图6（b）可知，当生成速率为4个请求/秒时，与U-Sarathi、U-Medusa和U-shape相比，HAT分别减少了TBT 31.9%、22.5%和41.3%。此外，随着生成率从 4 增加到



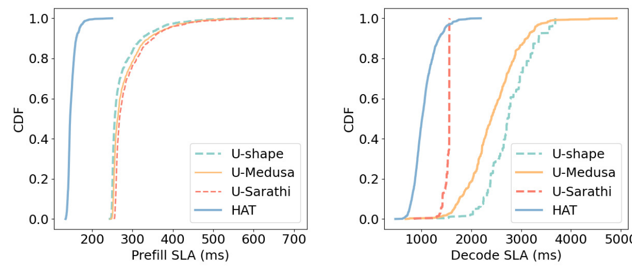
(a) SpecBench (b) CNN/DM

Fig. 8: Computation Delay with Standard Deviation.



(a) Prefill SLA (b) Decode SLA

Fig. 9: Compliance Rate with Different SLA on SpecBench.



(a) Prefill SLA (b) Decode SLA

Fig. 10: Compliance Rate with Different SLA on CNN/DM.

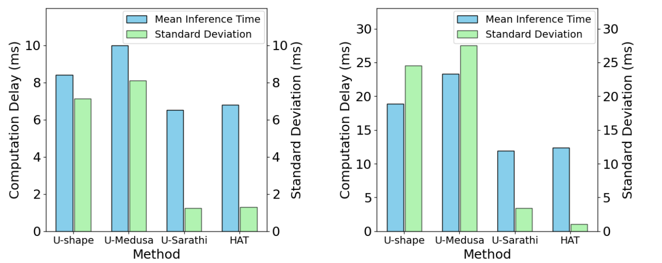
9 requests/s, the TBT for HAT and U-Sarathi increase only 5.3% and 7.8%, respectively, while U-Medusa and U-shape experience more substantial increases of 64.5% and 44%. HAT and U-Sarathi can mitigate the interference between multiple requests through chunking prompts, resulting in more stable TBT. However, HAT further reduces the TTFT and the TBT by parallel computation and communication along with its efficient speculative decoding. In contrast, the tree verification method employed by U-Medusa, which aggregates more candidate draft tokens to extend the accept length, tends to overload the server. Specifically, by Figure 7(a), at a generation rate of 4 requests/s on CNN/DM, HAT achieves the TTFT of 1027.4ms, while the TTFT of U-Sarathi, U-Medusa and U-shape is 1750.6ms, 2214.8ms and 2140.8ms, respectively. Besides, by Figure 7(b), with the same generation rate, HAT can reduce the TBT by 40.8%, 77.2%, and 74.7%, compared to the U-Sarathi, U-Medusa, and U-shape, respectively. These results demonstrate that HAT is effective under varying request loads.

Secondly, to further demonstrate the efficiency of HAT, we present the in-cloud computation delay of each GPU in Figure 8. Both HAT and U-Sarathi achieve stable and low computation delay by chunking long prompts. For instance, Figure 8(a) shows that the average computation delay of HAT on SpecBench is 6.8ms, while U-Sarathi, U-Medusa, and U-shape incur that of 6.5ms, 10.0ms, and 8.4ms, respectively. Besides, HAT maintains a standard deviation in computation delay of 1.3ms, significantly lower than the 8.1ms and 7.1ms observed for U-Medusa and U-shape, respectively, and slightly above the 1.2ms for U-Sarathi. U-Medusa and U-shape can not mitigate the impact of long prompts on the computation delay, leading to more volatile delays. Specifically, by Figure 8(b), the average computation delay for HAT on CNN/DM is 12.4ms with a standard deviation of 1.1ms, while U-Sarathi, U-Medusa, and U-shape exhibit the average computation delay of 11.9ms, 23.3ms, and 18.9ms, with corresponding standard deviations of 3.4ms, 27.5ms, and 24.6ms, respectively. These results illustrate that HAT is effective in ensuring stable inference performance.

Thirdly, to illustrate the advantage of HAT in maintaining the quality of service, we evaluate the SLA compliance rate, *i.e.*, the proportion of requests that satisfy the SLAs for *prefill* and *decode*. Concretely, the *prefill* SLA is defined as the delay for processing per 128 prompt tokens, and the *decode* SLA is defined as the delay for generating per 10 tokens in *decode* phase. With a specific request generation rate and the server's pipeline length of 1, we monitor the delay of each token generation to ascertain SLA compliance. The CDF plots displaying the SLA compliance rate with different SLA constraints for the four approaches are shown in Figures 9-10. By the results, HAT achieves the optimal compliance rate across most SLA cases. For instance, as illustrated in Figure 9(a), when the *prefill* SLA on SpecBench is 350ms, HAT reaches 100% of compliance rate, while the compliance rate of U-Sarathi, U-Medusa, and U-shape is 76.8%, 78.8% and 86.6%, respectively. Besides, by Figure 9(b), 50% of requests in HAT meet the *decode* SLA of 489.3ms, while 50% of requests in U-Sarathi, U-Medusa, and U-shape can satisfy more relaxed *decode* SLAs of 565.0ms, 659.8ms, and 786.1ms, respectively. HAT significantly reduces the TTFT through the prompt chunking mechanism, and its advantage is more obvious for requests with long prompts. Sarathi maintains a stable decoding rate through chunking, but it does not parallel transmission and computation, resulting in a longer delay in *prefill* phase. Specifically, as illustrated in Figure 10(a), when the *prefill* SLA on CNN/DM is 300ms, HAT maintains a 100% compliance rate, while U-Sarathi, U-Medusa, and U-shape exhibit lower rates of 64.7%, 68.9%, and 76.3%, respectively. Besides, HAT can outperform other baselines in most *decode* SLA cases through efficient speculative decode, by Figure 10(b), 90% of requests in HAT meet a *decode* SLA of 1352.7ms, while 90% of requests in U-Sarathi, U-Medusa, and U-shape satisfy more relaxed *decode* SLAs of 1561.5ms, 3109.6ms, and 3357.8ms, respectively.

4.3 SD Performance Evaluation

In this subsection, we evaluate the effectiveness of speculative decoding of HAT and U-Medusa across two datasets. We use the U-shape as a baseline to illustrate the speedup in *decode* phase, and we also compare the accept length (*i.e.*, number of accepted draft tokens) and the number of trained parameters. In this experiment, we use only one device collaborating with the server to eliminate wait delay and prevent interference from other devices. As shown in



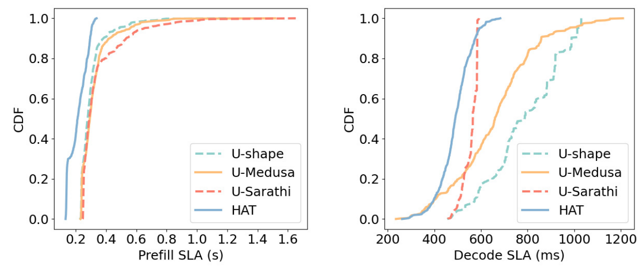
(a) SpecBench (b) CNN/DM

图 8：具有标准偏差的计算延迟。

在 9 个请求/秒的情况下，HAT 和 U-Sarathi 的 TBT 仅分别增加了 5.3% 和 7.8%，而 U-Medusa 和 U-shape 的 TBT 增幅更为大幅，达到 64.5% 和 44%。 HAT 和 U-Sarathi 可以通过分块提示来减轻多个请求之间的干扰，从而使 TBT 更加稳定。然而，HAT 通过并行计算和通信及其高效的推测解码进一步减少了 TTFT 和 TBT。相比之下，U-Medusa 采用的树验证方法聚合了更多候选草稿令牌以延长接受长度，这往往会使服务器过载。具体来说，由图7(a)可知，在CNN/DM上4个请求/秒的生成速率下，HAT实现了1027.4ms的TTFT，而U-Sarathi、U-Medusa和U-shape的TTFT分别为1750.6ms、2214.8ms和2140.8ms。此外，由图7(b)可知，在相同的发电率下，与U-Sarathi、U-Medusa和U-shape相比，HAT可分别降低TBT 40.8%、77.2%和74.7%。这些结果表明 HAT 在不同的请求负载下是有效的。

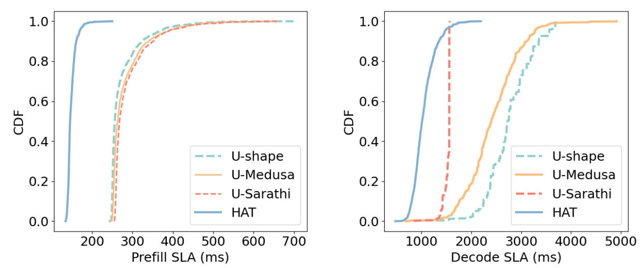
其次，为了进一步证明 HAT 的效率，我们在图 8 中展示了每个 GPU 的云端计算延迟。HAT 和 U-Sarathi 都通过对长提示进行分块来实现稳定且低的计算延迟。例如，图8（a）显示HAT在SpecBench上的平均计算延迟为6.8ms，而U-Sarathi、U-Medusa和U-shape的平均计算延迟分别为6.5ms、10.0ms和8.4ms。此外，HAT 的计算延迟标准偏差为 1.3ms，明显低于 U-Medusa 和 U-shape 的 8.1ms 和 7.1ms，略高于 U-Sarathi 的 1.2ms。 U- Medusa 和 U-shape 无法缓解长提示对计算延迟的影响，导致更不稳定的延迟。具体来说，由图8(b)可知，HAT在CNN/DM上的平均计算延迟为12.4ms，标准差为1.1ms，而U-Sarathi、U-Medusa和U-shape的平均计算延迟为11.9ms、23.3ms和18.9ms，相应的标准差为3.4ms、27.5ms和18.9ms。分别为 24.6 毫秒。这些结果说明 HAT 可以有效确保稳定的推理性能。

第三，为了说明 HAT 在维持服务质量方面的优势，我们评估 SLA 合规率 *i.e.*，即满足 *prefill* 和 *decode* SLA 的请求比例。具体地，*prefill*SLA被定义为每128个提示令牌的处理延迟，而*decode*SLA被定义为*decode*阶段每10个令牌的生成延迟。在特定的请求生成率和服务器管道长度为 1 的情况下，我们监控每个令牌生成的延迟以确定 SLA 合规性。中非发展基金



(a) Prefill SLA (b) Decode SLA

Fig. 9: Compliance Rate with Different SLA on SpecBench.



(a) Prefill SLA (b) Decode SLA

图 10：CNN/DM 上不同 SLA 的合规率。

plots displaying the SLA compliance rate with different SLA constraints for the four approaches are shown in Figures 9- 10. By the results, HAT achieves the optimal compliance rate across most SLA cases. For instance, as illustrated in Figure 9(a), when the *prefill* SLA on SpecBench is 350ms, HAT reaches 100% of compliance rate, while the compliance rate of U-Sarathi, U-Medusa, and U-shape is 76.8%, 78.8% and 86.6%, respectively. Besides, by Figure 9(b), 50% of requests in HAT meet the *decode* SLA of 489.3ms, while 50% of requests in U-Sarathi, U-Medusa, and U-shape can satisfy more relaxed *decode* SLAs of 565.0ms, 659.8ms, and 786.1ms, respectively. HAT通过提示分块机制显着降低了TTFT，对于提示较长的请求其优势更加明显。 Sarathi maintains a stable decoding rate through chunking, but it does not parallel transmission and computation, resulting in a longer delay in *prefill* phase. Specifically, as illustrated in Figure 10(a), when the *prefill* SLA on CNN/DM is 300ms, HAT maintains a 100% compliance rate, while U-Sarathi, U- Medusa, and U-shape exhibit lower rates of 64.7%, 68.9%, and 76.3%, respectively. Besides, HAT can outperform other baselines in most *decode* SLA cases through efficient speculative decode, by Figure 10(b), 90% of requests in HAT meet a *decode* SLA of 1352.7ms, while 90% of requests in U-Sarathi, U-Medusa, and U-shape satisfy more relaxed *decode* SLAs of 1561.5ms,分别为 3109.6ms 和 3357.8ms。

4.3 SD性能评估

在本小节中，我们评估两个数据集上 HAT 和 U-Medusa 的推测解码的有效性。我们使用 U 形作为基线来说明 *decode* 阶段的加速，并且我们还比较了接受长度（*i.e.*，接受的草案令牌的数量）和训练参数的数量。在本实验中，我们仅使用一台设备与服务器协作，以消除等待延迟并防止其他设备的干扰。如图所示

TABLE 4: SD Performance Evaluation.

Dataset	Method	Params	Accept	Speedup
SpecBench (Vicuna-7B)	U-shape	N/A	1.00	1.00×
	U-Medusa	591M	1.89	1.41×
	HAT	67M	2.06	1.65×
CNN/DM (Vicuna-13B)	U-shape	N/A	1.00	1.00×
	U-Medusa	760M	1.75	1.45×
	HAT	105M	1.98	1.60×

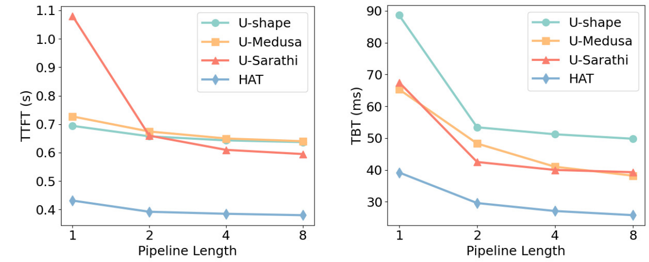
TABLE 5: Effect of Key Strategies.

Dataset	SD	PC	PD	TTFT(ms)	TBT(ms)
SpecBench (Vicuna-7B)	×	×	×	655.6	52.3
	×	✓	×	384.0	39.4
	✓	×	×	655.3	38.3
	✓	×	✓	657.9	33.8
	✓	✓	×	384.3	30.6
	✓	✓	✓	384.2	26.4
CNN/DM (Vicuna-13B)	×	×	×	1989.0	128.1
	×	✓	×	1038.7	74.2
	✓	×	×	1993.5	84.3
	✓	×	✓	1992.1	74.9
	✓	✓	×	1039.1	49.3
	✓	✓	✓	1039.9	43.5

Table 4, HAT consistently outperformed U-Medusa. For example, the HAT on SpecBench has an accept length of 2.06, and achieves a decoding speedup of 1.65× compared to U-shape, while the accept length and speedup of U-Medusa are only 1.89 and 1.41×, respectively. Besides, the HAT network has more efficient trained parameters, requiring only 67M parameters for training compared to U-Medusa's 591M parameters. The tree verification method of U-Medusa leads to a higher communication delay compared to HAT's threshold-based method. Concretely, for Vicuna-13B on CNN/DM, HAT achieves 1.60× speedup compared to U-shape, and its accept length is 1.98 with only 105M trained parameters, while U-Medusa needs to train 760M parameters and its accept length is only 1.75 and achieves 1.45× speedups. These results further demonstrate the advantage of HAT with efficient speculative decoding.

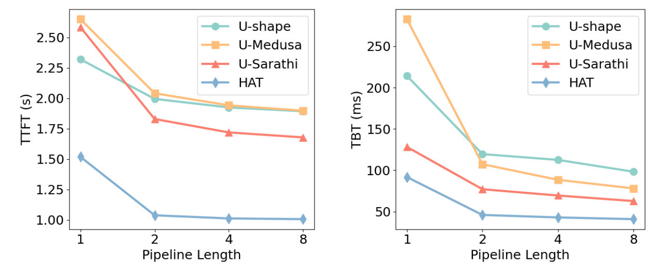
4.4 Effect of Key Strategies

There are three key strategies of HAT, speculative decoding (SD), prompt chunking (PC) and parallel drafting (PD), being developed to enhance the performance of U-shaped inference. Herein, we conduct several sets of experiments to evaluate the effectiveness of these critical strategies. We adopt the U-shape inference with some of these strategies as baselines. By Table 5, SD plays a significant role in reducing the TBT. The PC mechanism significantly reduces the TTFT by processing the transmission and computation in parallel. Besides, the PC mechanism can also reduce the TBT through the chunking operation. Furthermore, the PD mechanism further reduces the TBT by paralleling the drafting and verification stages of the SD. For example, compared to the U-shape inference on SpecBench, the introduction of SD can reduce TBT by 26.8%. When further combined with the PC mechanism, both the TTFT and TBT are reduced by 41.4% and 41.5%, respectively. Furthermore, the PD mechanism can further reduce the TBT by 49.5%. Similarly, the TTFT and TBT of U-shape inference on CNN/DM are 1989.0ms and 128.1ms on CNN/DM, respectively. The implementation of SD alone reduces the TBT to 84.3ms. When the PC mechanism is applied independently, it achieves the TTFT of 1038.7ms and the TBT of 74.2ms. Moreover, when SD and PC are utilized together without PD, HAT manages to decrease the TBT further by 61.5%. The results reflect the positive roles of speculative decoding, prompt chunking, and parallel drafting mechanisms in HAT.



(a) TTFT (b) TBT

Fig. 11: Delay with Different Pipeline Length on SpecBench.



(a) TTFT (b) TBT

Fig. 12: Delay with Different Pipeline Length on CNN/DM.

4.5 Effect of Pipeline Length Scale

To demonstrate the robustness of HAT, we evaluate the performance of HAT and baselines with different pipeline lengths in the server. Increasing the server's pipeline length can effectively reduce waiting delay as each GPU has less computation delay. As shown in Figures 11-12, both the TTFT and TBT of all approaches decrease as pipeline length increases. However, HAT consistently outperforms the other approaches. At a minimal pipeline length of 1, request accumulation within the server leads to increased delay, as incoming requests cannot be processed promptly. Besides, U-Sarathi increases the number of inference steps in *prefill* phase by chunking, leading to high TTFT. Specifically, by Figure 11, with a pipeline length of 1 on SpecBench, HAT achieves the TTFT of 431.4ms and the TBT of 39.2ms, while the TTFT for U-Sarathi, U-Medusa, and U-shape measure 1080.0ms, 727.1ms, and 694.0ms respectively, with corresponding TBT of 67.5ms, 65.3ms, and 88.6ms. Moreover, at a pipeline length of 4 on CNN/DM, Figure 12 demonstrates that HAT reduces the TTFT by about 36.9%, 40.7%, and 40.1%, while reduces the TBT by about 32.3%, 33.9%, and 47.1%, compared to U-Sarathi, U-Medusa, and U-shape, respectively. These results further illustrate the robustness of HAT.

5 Related Work

5.1 Cloud-based LLM Inference

Commercial cloud-based LLM systems typically optimize for high throughput by processing multiple concurrent requests from diverse devices. To address this requirement,

表 4：SD 性能评估。

Dataset	Method	Params	Accept	Speedup
SpecBench (Vicuna-7B)	U-shape	N/A	1.00	1.00×
	U-Medusa	591M	1.89	1.41×
	HAT	67M	2.06	1.65×
CNN/DM (Vicuna-13B)	U-shape	N/A	1.00	1.00×
	U-Medusa	760M	1.75	1.45×
	HAT	105M	1.98	1.60×

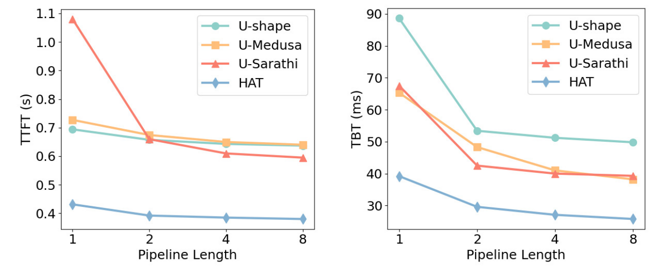
表 5：关键策略的效果。

Dataset	SD	PC	PD	TTFT(ms)	TBT(ms)
SpecBench (Vicuna-7B)	×	×	×	655.6	52.3
	×	✓	×	384.0	39.4
	✓	×	×	655.3	38.3
	✓	×	✓	657.9	33.8
	✓	✓	×	384.3	30.6
	✓	✓	✓	384.2	26.4
CNN/DM (Vicuna-13B)	×	×	×	1989.0	128.1
	×	✓	×	1038.7	74.2
	✓	×	×	1993.5	84.3
	✓	×	✓	1992.1	74.9
	✓	✓	×	1039.1	49.3
	✓	✓	✓	1039.9	43.5

表 4，HAT 的表现始终优于 U-Medusa。例如，SpecBench 上的 HAT 的接受长度为 2.06，与 U-shape 相比，实现了 1.65× 的解码加速，而 U-Medusa 的接受长度和加速分别仅为 1.89 和 1.41×。此外，HAT网络具有更高效的训练参数，与U-Medusa的591M参数相比，仅需要67M参数进行训练。与 HAT 基于阈值的方法相比，U-Medusa 的树验证方法导致了更高的通信延迟。具体来说，对于 CNN/DM 上的 Vicuna-13B，与 U-shape 相比，HAT 实现了 1.60× 加速，其接受长度为 1.98，仅需要 105M 训练参数，而 U-Medusa 需要训练 760M 参数，其接受长度仅为 1.75，实现了 1.45× 加速。这些结果进一步证明了 HAT 具有高效推测解码的优势。

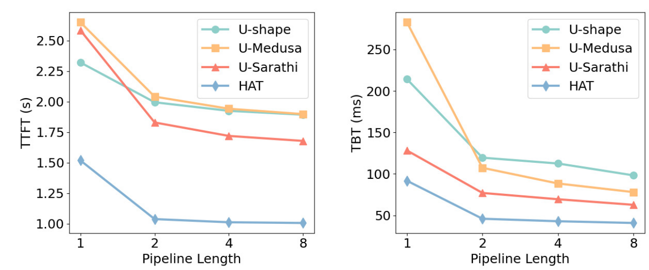
4.4 重点策略效果

HAT 的三个关键策略是推测解码（SD）、提示分块（PC）和并行起草（PD），正在开发中以增强 U 形推理的性能。在这里，我们进行了几组实验来评估这些关键策略的有效性。我们采用 U 形推理，其中一些策略作为基线。从表 5 可以看出，SD 在降低 TBT 方面发挥着重要作用。 PC机制通过并行处理传输和计算，显着减少了TTFT。此外，PC机制还可以通过分块操作来减少TBT。此外，PD机制通过并行SD的起草和验证阶段，进一步减少了TBT。例如，与SpecBench上的U型推理相比，SD的引入可以将TBT降低26.8%。当进一步与PC机制结合时，TTFT和TBT分别减少了41.4%和41.5%。此外，PD机制还可进一步降低TBT 49.5%。同样，CNN/DM 上 U 形推理的 TTFT 和 TBT 在 CNN/DM 上分别为 1989.0ms 和 128.1ms。仅执行 SD 即可将 TBT 降低至 84.3ms。当单独应用PC机制时，实现了1038.7ms的TTFT和74.2ms的TBT。此外，当SD



(a) TTFT (b) TBT

图 11：SpecBench 上不同管道长度的延迟。



(a) TTFT (b) TBT

图 12：CNN/DM 上不同管道长度的延迟。

与 PC 一起使用，无需 PD，HAT 成功将 TBT 进一步降低了 61.5%。结果反映了 HAT 中推测解码、提示分块和并行起草机制的积极作用。

4.5 管道长度规模的影响

为了证明 HAT 的稳健性，我们评估了服务器中不同管道长度的 HAT 和基线的性能。增加服务器的管道长度可以有效减少等待延迟，因为每个 GPU 的计算延迟较小。如图 11-12 所示，所有方法的 TTFT 和 TBT 都随着管道长度的增加而减小。然而，HAT 始终优于其他方法。当最小管道长度为 1 时，服务器内的请求累积会导致延迟增加，因为传入的请求无法立即得到处理。此外，U-Sarathi 通过分块增加了 *prefill* 阶段的推理步骤数，从而导致高 TTFT。具体来说，由图11可知，在SpecBench上管道长度为1时，HAT的TTFT为431.4ms，TBT为39.2ms，而U-Sarathi、U-Medusa和U-shape的TTFT分别为 1080.0ms、727.1ms和694.0ms，相应的TBT为67.5ms， 65.3 毫秒和 88.6 毫秒。此外，在 CNN/DM 上的管道长度为 4 时，图 12 表明，与 U-Sarathi、U-Medusa 和 U-shape 相比，HAT 分别将 TTFT 减少了约 36.9%、40.7% 和 40.1%，同时将 TBT 减少了约 32.3%、33.9% 和 47.1%。这些结果进一步说明了 HAT 的稳健性。

5 相关工作

5.1 基于云的LLM推理

基于云的商业大语言模型系统通常通过处理来自不同设备的多个并发请求来优化高吞吐量。为了满足这一要求，

Orca [20] introduces continuous batching, which dynamically integrates and removes requests during forward passes without padding or batch completion constraints, significantly improving system throughput. Building on this concept, vLLM [41] combines continuous batching with priority scheduling, particularly prioritizing the *prefill* phase. While effective, this prioritization can interfere with ongoing decoding processes, potentially compromising decoding rates due to the computationally intensive nature of *prefill* phase, which introduces delays in *decode* phase. Therefore, Sarathi-Serve [21] divides the prompt to fully utilize the computational resources and reduce computation delay for *decode* phase. In contrast to these existing approaches, our framework leverages local device computational resources while simultaneously minimizing device-cloud communication latency, offering a distinct advantage in device-cloud collaborative inference scenarios.

5.2 Speculative Decoding

Speculative decoding has emerged as a pivotal technique in device-cloud collaborative inference frameworks to address the autoregressive limitations of LLMs. For example, Hao *et al.* [10] integrate speculative decoding into the device-cloud framework, and systematically analyze how acceptance thresholds influence both inference accuracy and speed. Similarly, Oh *et al.* [11] develop a hybrid framework that leverages on-device SLMs and the in-cloud LLM, utilizing the uncertainty of SLM to dynamically adjust LLM acceptance probabilities and verification processes, thus achieving optimal generation speed with negligible accuracy degradation. To further refine speculative decoding, Medusa [25] introduces a self-drafting mechanism to minimize drafting overhead while employing tree-based verification to maximize accepted sequence lengths. Concurrently, Eagle [24] enhances acceptance probabilities through a specialized autoregressive head trained via feature-layer knowledge distillation. However, these methods involve sharing input and output tokens, posing potential privacy risks.

5.3 Split Inference

Split inference has emerged as a privacy-preserving framework for device-cloud inference. For example, Sa *et al.* [16] employ U-shaped inference to safeguard both input and output data from exposure beyond devices. Similarly, Ohta *et al.* [15] resides LLM's middle submodel in the cloud, enabling computation offloading while protecting privacy. Based on split inference, Mai *et al.* [14] add noise to intermediate data, enhancing privacy at the cost of some inference accuracy. Moreover, Shen *et al.* [42] divides the LLM and then fine-tunes the on-device submodel with private data, which meets individual demand while enhancing privacy. However, these methods do not address the communication delay issue due to the massive and frequent transmission of hidden states.

6 Conclusions

In this paper, we propose a novel device-cloud collaborative inference framework, named HAT, which integrates the advantages of U-shaped inference and speculative decoding to provide LLM services with low delay, high accuracy, and enhanced privacy. Besides, we propose a prompt chunking mechanism and a parallel drafting mechanism to accelerate inference speed. The experimental results show that the HAT can reduce the TTFT by 41% to 54% and the TBT by 41% to 77%, compared to the baselines.

References

[1] C. Wang, B. Zhang, D. Sui, Z. Tum, X. Liu, and J. Kang, "A survey on effective invocation methods of massive llm services," *arXiv e-prints*, pp. arXiv–2402, 2024.

[2] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Pre-trained language models for text generation: A survey," *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–39, 2024.

[3] F. Duarte. (2023) Number of chatgpt users (nov 2023). Accessed: Nov 24, 2023. [Online]. Available: https://explodingtopics.com/blog/chatgpt-users

[4] C. Zhang, M. Yu, W. Wang, and F. Yan, "{MArk}: Exploiting cloud services for {Cost-Effective},{SLO-Aware} machine learning inference serving," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 1049–1062.

[5] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun *et al.*, "Personal llm agents: Insights and survey about the capability, efficiency and security," *arXiv preprint arXiv:2401.05459*, 2024.

[6] C. Cui, Y. Ma, X. Cao, W. Ye, Y. Zhou, K. Liang, J. Chen, J. Lu, Z. Yang, K.-D. Liao *et al.*, "A survey on multimodal large language models for autonomous driving," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 958–979.

[7] G. De Vito, "Assessing healthcare software built using iot and llm technologies," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 476–481.

[8] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large language models (llms) inference offloading and resource allocation in cloud-edge computing: An active inference approach," *IEEE Transactions on Mobile Computing*, 2024.

[9] T. Gunter, Z. Wang, C. Wang, R. Pang, A. Narayanan, A. Zhang, B. Zhang, C. Chen, C.-C. Chiu, D. Qiu *et al.*, "Apple intelligence foundation language models," *arXiv preprint arXiv:2407.21075*, 2024.

[10] Z. Hao, H. Jiang, S. Jiang, J. Ren, and T. Cao, "Hybrid slm and llm for edge-cloud collaborative inference," in *Proceedings of the Workshop on Edge and Mobile Foundation Models*, 2024, pp. 36–41.

[11] S. Oh, J. Kim, J. Park, S.-W. Ko, T. Q. Quek, and S.-L. Kim, "Uncertainty-aware hybrid inference with on-device small and remote large language models," *arXiv preprint arXiv:2412.12687*, 2024.

[12] W. Zheng, Y. Chen, W. Zhang, S. Kundu, Y. Li, Z. Liu, E. P. Xing, H. Wang, and H. Yao, "Citer: Collaborative inference for efficient large language model decoding with token-level routing," *arXiv preprint arXiv:2502.01976*, 2025.

[13] B. Yan, K. Li, M. Xu, Y. Dong, Y. Zhang, Z. Ren, and X. Cheng, "On protecting the data privacy of large language models (llms): A survey," *arXiv preprint arXiv:2403.05156*, 2024.

[14] P. Mai, R. Yan, Z. Huang, Y. Yang, and Y. Pang, "Split-and-denoise: Protect large language model inference with local differential privacy," *arXiv preprint arXiv:2310.09130*, 2023.

[15] S. Ohta and T. Nishio, "Lambda-split: A privacy-preserving split computing framework for cloud-powered generative ai," *arXiv preprint arXiv:2310.14651*, 2023.

[16] C.-C. Sa, L.-C. Cheng, H.-H. Chung, T.-C. Chiu, C.-Y. Wang, A.-C. Pang, and S.-T. Chen, "Ensuring bidirectional privacy on wireless split inference systems," *IEEE Wireless Communications*, 2024.

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[18] "Openai gpt-3: Understanding the architecture." https://www.theaidream.com/post/openai-gpt-3-understanding-the-architecture.

[19] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

Orca [20] 引入了连续批处理，它在前向传递过程中动态地集成和删除请求，而无需填充或批处理完成约束，从而显着提高了系统吞吐量。在此概念的基础上，vLLM [41] 将连续批处理与优先级调度相结合，特别是优先考虑 *prefill* 阶段。虽然有效，但这种优先级可能会干扰正在进行的解码过程，由于 *prefill* 阶段的计算密集性，可能会影响解码率，这会在 *decode* 阶段引入延迟。因此，Sarathi-Serve [21]将提示划分为 *decode*阶段，以充分利用计算资源并减少计算延迟。与这些现有方法相比，我们的框架利用本地设备计算资源，同时最大限度地减少设备-云通信延迟，在设备-云协作推理场景中提供明显的优势。

5.2 推测解码

推测性解码已成为设备-云协作推理框架中的一项关键技术，旨在解决大语言模型的自回归限制。例如，Hao *et al.* [10]将推测解码集成到端云框架中，并系统地分析接受阈值如何影响推理精度和速度。同样，Oh *et al.* [11]开发了一个混合框架，利用设备上的SLM和云中的LLM，利用SLM的不确定性来动态调整LLM的接受概率和验证过程，从而在精度下降可忽略不计的情况下实现最佳的生成速度。为了进一步完善推测解码，Medusa [25]引入了一种自起草机制来最小化起草开销，同时采用基于树的验证来最大化可接受的序列长度。同时，Eagle [24] 通过通过特征层知识蒸馏训练的专门自回归头来提高接受概率。然而，这些方法涉及共享输入和输出令牌，带来潜在的隐私风险。

5.3 分割推理

分割推理已成为设备云推理的隐私保护框架。例如，Sa *et al.* [16] 采用 U 形推理来保护输入和输出数据免遭设备之外的泄露。同样，Ohta *et al.* [15]将LLM的中间子模型驻留在云端，在保护隐私的同时实现计算卸载。基于分割推理，Mai *et al.* [14]向中间数据添加噪声，以牺牲一些推理准确性为代价来增强隐私性。此外，Shen*et al.*[42]对LLM进行了划分，然后用私有数据对设备上的子模型进行微调，在满足个人需求的同时增强了隐私性。然而，这些方法并没有解决由于隐藏状态的大量且频繁的传输而导致的通信延迟问题。

6 结论

在本文中，我们提出了一种新颖的端云协同推理框架，称为HAT，它集成了U型推理和推测解码的优点提供低延迟、高精度、增强隐私性的LLM服务。此外，我们提出了快速分块机制和并行起草机制来加快推理速度。实验结果表明，与基线相比，HAT 可使 TTFT 降低 41% 至 54%，TBT 降低 41% 至 77%。

参考

[1] C. Wang，B. Zhang，D. Sui，Z. Tum，X. Liu，J. Kang，“大规模 llm 服务有效调用方法的调查”，*arXiv e-prints*，第 arXiv–2402，2024。[2] J. Li，T. Tang，W. X. Zhao，J.-Y.聂，J.-R。Wen，“用于文本生成的预训练语言模型：一项调查”，*ACM Computing Surveys*，卷。56，没有。9，第 1-39 页，2024 年。[3] F. Duarte。(2023) chatgpt 用户数量（2023 年 11 月）。访问时间：2023 年 11 月 24 日。[在线]。可用：https://explodingtopics.com/blog/chatgpt-users [4] C.Zhang、M.Yu、W.Wang 和 F.Yan，“{MArk}：利用云服务实现{成本有效}、{SLO 感知}机器学习推理服务”，载于 *2019 USENIX Annual Technical Conference (USENIX ATC 19)*，2019 年，第 1049–1062 页。[5] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun *et al.*，“个人llm代理：关于能力、效率和安全性的见解和调查”，*arXiv preprint arXiv:2401.05459*，2024年。[6] C. Cui，Y. Ma，X. Cao，W. Ye，Y. Zhou，梁K.，陈J.，卢J.，杨Z.，K.-D。Liao *et al.*，“自动驾驶多模式大语言模型的调查”，*Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*，2024 年，第 958–979 页。[7] G. De Vito，“评估使用物联网和 llm 技术构建的医疗保健软件”，*Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*，2024 年，第 476–481 页。[8] Y. He、J. Fang、F. R. Yu 和 V. C. Leung，“云边缘计算中的大语言模型 (llms) 推理卸载和资源分配：一种主动推理方法”，*IEEE Transactions on Mobile Computing*，2024 年。[9] T. Gunter、Z. Wang、C. Wang、R. Pang、A. Narayanan、A. Zhu、B. Zhu、C. Chen、C.-C. Chiu, D. Qiu *et al.*，“Apple 智能基础语言模型”，*arXiv preprint arXiv:2407.21075*，2024 年。[10] Z.hao、H. Jiang、S. Jiang、J. Ren 和 T. Cao，“用于边云协作推理的混合 slm 和 llm”，*Proceedings of the Workshop on Edge and Mobile Foundation Models*，2024 年，第 36-41 页。[11] S. Oh、J. Kim、J. Park、S.-W。Ko，T. Q. Quek，和 S.-L。Kim，“设备上小型和远程大语言模型的不确定性感知混合推理”，*arXiv preprint arXiv:2412.12687*，2024 年。[12] W. Cheng、Y. Chen、W. Zhu、S. Kundu、Y. Li、Z. Liu、E. P. Xing、H. Wang 和 H. Yao，“Citer：通过令牌级路由进行高效大语言模型解码的协作推理”，*arXiv preprint arXiv:2502.01976*，2025。[13] B. Yan、K. Li、M. Xu、Y. Dong、Y. Zhu、Z. Ren 和 X. Cheng，“关于保护大语言模型 (llms) 的数据隐私：一项调查”，*arXiv preprint arXiv:2403.05156*，2024。[14] P. Mai、R. Yan、Z. Huang、Y. Yang 和 Y. Pang，“分割和去噪：保护大语言模型局部差分隐私推理”，*arXiv preprint arXiv:2310.09130*，2023 年。[15] S. Ohta 和 T. Nishio，“Lambda-split：用于云驱动的生成人工智能的隐私保护分割计算框架”，*arXiv preprint arXiv:2310.14651*，2023 年。[16] C.-C. Sa，L.-C。郑，H.-H。钟，T.-C。邱，C.-Y。王，A.-C。庞和S.-T。Chen，“确保无线分割推理系统的双向隐私”，*IEEE Wireless Communications*，2024 年。[17] A. Vaswani、N. Shazeer、N. Parmar、J. Uszkoreit、L. Jones、A. N. Gomez、⊠。Kaiser 和 I. Polosukhin，“注意力就是你所需要的”，*Advances in neural information processing systems*，卷。2017 年 3 月 30 日。[18]“Openai gpt-3：了解架构。”https://www.theaidream.com/post/openai-gpt-3-understanding-the-architecture。[19] H. Touvron、L. Martin、K. Stone、P. Albert、A. Almahairi、Y. Babaei、N. Bashlykov、S. Batra、P. Bhargava、S. Bhosale *et al.*，“Llama 2：开放基础和微调聊天模型”，*arXiv preprint arXiv:2307.09288*，2023 年。

[20] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[21] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *arXiv preprint arXiv:2403.02310*, 2024.

[22] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko *et al.*, "Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," *arXiv preprint arXiv:2401.08671*, 2024.

[23] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.

[24] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle: Speculative sampling requires rethinking feature uncertainty," *arXiv preprint arXiv:2401.15077*, 2024.

[25] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple llm inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.

[26] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality," *See https://vicuna. lmsys. org (accessed 14 April 2023)*, vol. 2, no. 3, p. 6, 2023.

[27] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings of the Association for Computational Linguistics ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand and virtual meeting: Association for Computational Linguistics, Aug. 2024, pp. 7655–7671. [Online]. Available: https://aclanthology.org/2024.findings-acl.456

[28] D. Leroy, A. Coucke, T. Lavril, T. Gisselbrecht, and J. Dureau, "Federated learning for keyword spotting," in *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2019, pp. 6341–6345.

[29] X. Lyu, C. Ren, W. Ni, H. Tian, R. P. Liu, and E. Dutkiewicz, "Optimal online data partitioning for geo-distributed machine learning in edge of wireless networks," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 10, pp. 2393–2406, 2019.

[30] D. Merkel *et al.*, "Docker: lightweight linux containers for consistent development and deployment," *Linux j*, vol. 239, no. 2, p. 2, 2014.

[31] N. Naik, "Building a virtual system of systems using docker swarm in multiple clouds," in *2016 IEEE International Symposium on Systems Engineering (ISSE)*. IEEE, 2016, pp. 1–3.

[32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[33] E. Gabriel, G. E. Fagg, G. Bosilca, T. Angskun, J. J. Dongarra, J. M. Squyres, V. Sahay, P. Kambadur, B. Barrett, A. Lumsdaine *et al.*, "Open mpi: Goals, concept, and design of a next generation mpi implementation," in *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*. Springer, 2004, pp. 97–104.

[34] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[35] Z. Ye, L. Chen, R. Lai, Y. Zhao, S. Zheng, J. Shao, B. Hou, H. Jin, Y. Zuo, L. Yin *et al.*, "Accelerating self-attentions for llm serving with flashinfer," *URL https://flashinfer. ai/2024/02/02/introduce-flashinfer. html*, 2024.

[36] "Nvidia collective communications library (nccl)," https://developer.nvidia.com/nccl.

[37] A. Tirumala, "Iperf: The tcp/udp bandwidth measurement tool," *http://dast. nlanr. net/Projects/Iperf/*, 1999.

[38] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1073–1083. [Online]. Available: https://www.aclweb.org/anthology/P17-1099

[39] D.-J. Han, D.-Y. Kim, M. Choi, D. Nickel, J. Moon, M. Chiang, and C. G. Brinton, "Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks," *IEEE Transactions on Mobile Computing*, 2023.

[40] "Sharegpt," https://huggingface.co/datasets/Aeala/ShareGPT_Vicuna_unfiltered, 2023.

[41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "vllm: Easy, fast, and cheap llm serving with pagedattention," *See https://vllm. ai/(accessed 9 August 2023)*, 2023.

[42] X. Shen, Y. Liu, H. Liu, J. Hong, B. Duan, Z. Huang, Y. Mao, Y. Wu, and D. Wu, "A split-and-privatize framework for large language model fine-tuning," *arXiv preprint arXiv:2312.15603*, 2023.

References

[1] C. Wang, B. Zhang, D. Sui, Z. Tum, X. Liu, and J. Kang, "A survey on effective invocation methods of massive llm services," *arXiv e-prints*, pp. arXiv–2402, 2024.

[2] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Pre-trained language models for text generation: A survey," *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–39, 2024.

[3] F. Duarte. (2023) Number of chatgpt users (nov 2023). Accessed: Nov 24, 2023. [Online]. Available: https://explodingtopics.com/blog/chatgpt-users

[4] C. Zhang, M. Yu, W. Wang, and F. Yan, "{MArk}: Exploiting cloud services for {Cost-Effective},{SLO-Aware} machine learning inference serving," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 1049–1062.

[5] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun *et al.*, "Personal llm agents: Insights and survey about the capability, efficiency and security," *arXiv preprint arXiv:2401.05459*, 2024.

[6] C. Cui, Y. Ma, X. Cao, W. Ye, Y. Zhou, K. Liang, J. Chen, J. Lu, Z. Yang, K.-D. Liao *et al.*, "A survey on multimodal large language models for autonomous driving," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 958–979.

[7] G. De Vito, "Assessing healthcare software built using iot and llm technologies," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 476–481.

[8] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large language models (llms) inference offloading and resource allocation in cloud-edge computing: An active inference approach," *IEEE Transactions on Mobile Computing*, 2024.

[9] T. Gunter, Z. Wang, C. Wang, R. Pang, A. Narayanan, A. Zhang, B. Zhang, C. Chen, C.-C. Chiu, D. Qiu *et al.*, "Apple intelligence foundation language models," *arXiv preprint arXiv:2407.21075*, 2024.

[10] Z. Hao, H. Jiang, S. Jiang, J. Ren, and T. Cao, "Hybrid slm and llm for edge-cloud collaborative inference," in *Proceedings of the Workshop on Edge and Mobile Foundation Models*, 2024, pp. 36–41.

[11] S. Oh, J. Kim, J. Park, S.-W. Ko, T. Q. Quek, and S.-L. Kim, "Uncertainty-aware hybrid inference with on-device small and remote large language models," *arXiv preprint arXiv:2412.12687*, 2024.

[12] W. Zheng, Y. Chen, W. Zhang, S. Kundu, Y. Li, Z. Liu, E. P. Xing, H. Wang, and H. Yao, "Citer: Collaborative inference for efficient large language model decoding with token-level routing," *arXiv preprint arXiv:2502.01976*, 2025.

[13] B. Yan, K. Li, M. Xu, Y. Dong, Y. Zhang, Z. Ren, and X. Cheng, "On protecting the data privacy of large language models (llms): A survey," *arXiv preprint arXiv:2403.05156*, 2024.

[14] P. Mai, R. Yan, Z. Huang, Y. Yang, and Y. Pang, "Split-and-denoise: Protect large language model inference with local differential privacy," *arXiv preprint arXiv:2310.09130*, 2023.

[15] S. Ohta and T. Nishio, "Lambda-split: A privacy-preserving split computing framework for cloud-powered generative ai," *arXiv preprint arXiv:2310.14651*, 2023.

[16] C.-C. Sa, L.-C. Cheng, H.-H. Chung, T.-C. Chiu, C.-Y. Wang, A.-C. Pang, and S.-T. Chen, "Ensuring bidirectional privacy on wireless split inference systems," *IEEE Wireless Communications*, 2024.

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[18] "Openai gpt-3: Understanding the architecture." https://www.theaidream.com/post/openai-gpt-3-understanding-the-architecture.

[20] G.-I。 Yu, J. S. Jeong, G.-W. Kim、S. Kim 和 B.-G。 Chun, “Orca：基于 {Transformer} 生成模型的分布式服务系统”， *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*，2022 年，第 521-538 页。 [21] A. Agrawal、N. Kedia、A. Panwar、J. Mohan、N. Kwatra、B. S. Gulavani、A. Tumanov 和 R. Ramjee, “利用 sarathi-serve 控制 llm 推理中的吞吐量-延迟权衡”， *arXiv preprint arXiv:2403.02310*，2024 年。 [22] C. Holmes、M. Tanaka、M. Wyatt、A. A. Awan, J. Rasley, S. Rajb-handari, R. Y. Aminabadi, H.qin, A. Bakhtiari, L. Kurilenko *et al.*, “Deepspeed-fastgen：通过 mii 和 deepspeed-inference 实现 llms 的高吞吐量文本生成”， *arXiv preprint arXiv:2401.08671*，2024 年。 [23] Y. Leviathan、M. Kalman 和 Y. Matias, “快速推理通过推测解码从 Transformer 中获取”， 在 *International Conference on Machine Learning* 中。 PMLR，2023，第 19 274–19 286 页。 [24] Y. Li、F. Wei、C. Zhang 和 H. Zhu, “Eagle：推测性采样需要重新思考特征不确定性”， *arXiv preprint arXiv:2401.15077*，2024。 [25] T. Cai、Y. Li、Z. Geng、H. Peng、J. D. Lee、D. Chen 和 T. Dao, “Medusa：具有多个解码头的简单 llm 推理加速框架”， *arXiv preprint arXiv:2401.10774*，2024 年。[26] W.-L。 Jiang, Z. Li, Z. Lin, Y. Shen, Z. Wu, H. Zhang, L. Cheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, “Vicuna：一款开源聊天机器人，以 90%* chatgpt 质量打动 gpt-4”， *See https://vicuna. lmsys. org (accessed 14 April 2023)*，卷。 2、没有。 3，第 3 页。 2023 年 6 月。[27] H. Xia、Z. Yang、Q. Dong、P. Wang、Y. Li、T. Ge、T. Liu、W. Li 和 Z. Sui, “解锁大语言模型推理的效率：推测性解码的全面调查”， *Findings of the Association for Computational Linguistics ACL 2024*，L.-W. Ku， A. Martins 和 V. Srikumar，编辑。泰国曼谷和虚拟会议：计算语言学协会，2024 年 8 月，第 7655–7671 页。 [在线的]。可用：https://aclanthology.org/2024.findings-acl.456 [28] D. Leroy、A. Coucke、T. Lavril、T. Gisselbrecht 和 J. Dureau, “关键字识别的联合学习”， 参见 *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*。 IEEE，2019 年，第 6341–6345 页。 [29] X. Lyu、C. Ren、W. Ni、H. Tian、R. P. Liu 和 E. Dutkiewicz, “无线网络边缘地理分布式机器学习的最佳在线数据分区”， *IEEE Journal on Selected Areas in Communications*，卷。 37、没有。 10，第 2393–2406 页，2019 年。 [30] D. Merkel *et al.*, “Docker：用于一致开发和部署的轻量级 Linux 容器”， *Linux j*，卷。 239，没有。 2，第 14 页。 2014 年 2 月 2 日。 [31] N. Naik, “在多个云中使用 docker swarm 构建系统的虚拟系统”，参见 *2016 IEEE International Symposium on Systems Engineering (ISSE)*。 IEEE，2016 年，第 1-3 页。 [32] A. Paszke、S. Gross、F. Massa、A. Lerer、J. Bradbury、G. Chanan、T. Killeen、Z. Lin、N. Gimelshein、L. Antiga *et al.*, “Pytorch：命令式高性能深度学习库”， *Advances in neural information processing systems*，卷。 2019 年 32 月。[33] E. Gabriel、G. E. Fagg、G. Bosilca、T. Angskun、J. J. Dongarra、J. M. Squyres、V. Sahay、P. Kambadur、B. Barrett、A. Lumsdaine *et al.*, “开放 mpi：下一代 mpi 实现的目标、概念和设计”， *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*。施普林格，2004 年，第 97-104 页。 [34] T. Dao, “Flashattention-2：更快的注意力，更好的并行性和工作划分”， *arXiv preprint arXiv:2307.08691*，2023。 [35] Z. Ye, L. Chen, R. Lai, Y. Zhu, S. Cheng, J. Shao, B. Hou, H. Jin, Y. Zuo, L. Yin *et al.*, “加速自我注意力，以实现 llm 服务flashinfer，” *URL https://flashinfer. ai/2024/02/02/introduce- flashinfer. html*，2024 年。[36]“Nvidia 集体通信库 (nccl)”， https://developer.nvidia.com/nccl。 [37] A. Tirumala, “Iperf：tcp/udp 带宽测量工具”， *http://dast. nlanr. net/Projects/Iperf/*，1999。 [38] A. See、P. J. Liu 和 C. D. Manning, “开门见山：用指针生成器网络进行总结”， 见 *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*。加拿大温哥华：计算语言学协会，2017 年 7 月，第 1073–1083 页。 [在线的]。可用：https://www.aclweb.org/anthology/P17-1099 [39] D.-J.汉， D.-Y。 Kim、M. Choi、D. Nickel、J. Moon、M. Jiang 和 C. G. Brinton, “具有联合个性化的联邦分割学习无线边缘网络推理阶段优化的泛化”， *IEEE Transactions on Mobile Computing*，2023 年。[40]“Sharegpt”， https://huggingface.co/datasets/Aeala/ShareGPT Vicuna unfiltered，2023 年。[41] W. Kwon、Z. Li、S. Zhuang、Y. Shen、L. Cheng、C. Yu、J. Gonzalez、H. Zhang 和 I. Stoica, “vllm：通过 pagedattention 实现简单、快速、廉价的 llm 服务”， *See https://vllm. ai/(accessed 9 August 2023)*，2023。 [42] X. Shen、Y. Liu、H. Liu、J. Hong、B. Duan、Z. Huang、Y. Mao、Y. Wu 和 D. Wu, “用于大语言模型微调的拆分和私有化框架”， *arXiv preprint arXiv:2312.15603*，2023。

参考

[1] C. Wang， B. Zhang， D. Sui， Z. Tum， X. Liu， J. Kang， “大规模 llm 服务有效调用方法的调查”， *arXiv e-prints*，第 arXiv–2402，2024。 [2] J. Li， T. Tang， W. X. Zhao， J.-Y.聂， J.-R。 Wen， “用于文本生成的预训练语言模型：一项调查”， *ACM Computing Surveys*，卷。 56，没有。 9，第 1-39 页，2024 年。 [3] F. Duarte。 (2023) chatgpt 用户数量（2023 年 11 月）。访问时间：2023 年 11 月 24 日。[在线]。可用：https://explodingtopics.com/blog/chatgpt-users [4] C.Zhang、M.Yu、W.Wang 和 F.Yan， “{MArk}：利用云服务实现{成本有效的}、{SLO 感知}机器学习推理服务”， 载于 *2019 USENIX Annual Technical Conference (USENIX ATC 19)*， 2019 年，第 1049–1062 页。 [5] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun *et al.*， “个人 llm 代理：关于能力、效率和安全性的见解和调查”， *arXiv preprint arXiv:2401.05459*，2024。 [6] C. Cui、Y. Ma、X. Cao、W. Ye、Y. Zhou，梁K.，陈J.，卢J.，杨Z.，K.-D。 Liao *et al.*， “自动驾驶多模式大语言模型的调查”， *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*，2024 年，第 958–979 页。 [7] G. De Vito， “评估使用物联网和 LLM 技术构建的医疗保健软件”， *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*，2024 年，第 476–481 页。 [8] Y. He、J. Fang、F. R. Yu 和 V. C. Leung， “云边缘计算中的大语言模型 (llms) 推理卸载和资源分配：一种主动推理方法”， *IEEE Transactions on Mobile Computing*，2024 年。 [9] T. Gunter、Z. Wang、C. Wang、R. Pang、A. Narayanan、A. Zhu、B. Zhu、C. Chen、C.-C. Chiu, D. Qiu *et al.*， “Apple 智能基础语言模型”， *arXiv preprint arXiv:2407.21075*，2024 年。 [10] Z.hao、H. Jiang、S. Jiang、J. Ren 和 T. Cao， “用于边云协作推理的混合 slm 和 llm”， *Proceedings of the Workshop on Edge and Mobile Foundation Models*，2024 年，第 36-41 页。 [11] S. Oh、J. Kim、J. Park、S.-W。 Ko， T. Q. Quek，和 S.-L。 Kim， “设备上小型和远程大语言模型的不确定性感知混合推理”， *arXiv preprint arXiv:2412.12687*，2024 年。[12] W. Cheng、Y. Chen、W. Zhu、S. Kundu、Y. Li、Z. Liu、E. P. Xing、H. Wang 和 H. Yao， “Citer：使用令牌级路由进行高效大语言模型解码的协作推理”， *arXiv preprint arXiv:2502.01976*，2025。[13] B. Yan， K. Li， M. Xu， Y. Dong， Y. Zhang， Z. Ren 和 X. Cheng， “关于保护大语言模型（llms）的数据隐私：一项调查”， *arXiv preprint arXiv:2403.05156*，2024。[14] P. Mai， R. Yan， Z. Huang， Y. Yang， 和 Y. Pang， “分割和去噪：保护大语言模型局部差分隐私的推理”， *arXiv preprint arXiv:2310.09130*，2023 年。[15] S. Ohta 和 T. Nishio， “Lambda-split：用于云驱动的生成人工智能的隐私保护分割计算框架”， *arXiv preprint arXiv:2310.14651*，2023 年。[16] C.-C. Sa， L.-C。郑， H.-H。钟， T.-C。邱， C.-Y。王， A.-C。庞和S.-T。 Chen， “确保无线分割推理系统的双向隐私”， *IEEE Wireless Communications*，2024 年。 [17] A. Vaswani、N. Shazeer、N. Parmar、J. Uszkoreit、L. Jones、A. N. Gomez、⊠. Kaiser 和 I. Polosukhin， “你所需要的就是注意力”， *Advances in neural information processing systems*，卷。 2017 年 3 月 30 日。[18]“Openai gpt-3：了解架构。” https://www.theaidream.com/post/openai-gpt-3-understanding-the-architecture。

[19] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[20] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[21] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *arXiv preprint arXiv:2403.02310*, 2024.

[22] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko *et al.*, "Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," *arXiv preprint arXiv:2401.08671*, 2024.

[23] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.

[24] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle: Speculative sampling requires rethinking feature uncertainty," *arXiv preprint arXiv:2401.15077*, 2024.

[25] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple llm inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.

[26] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality," *See https://vicuna. lmsys. org (accessed 14 April 2023)*, vol. 2, no. 3, p. 6, 2023.

[27] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings of the Association for Computational Linguistics ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand and virtual meeting: Association for Computational Linguistics, Aug. 2024, pp. 7655–7671. [Online]. Available: https://aclanthology.org/2024.findings-acl.456

[28] D. Leroy, A. Coucke, T. Lavril, T. Gisselbrecht, and J. Dureau, "Federated learning for keyword spotting," in *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2019, pp. 6341–6345.

[29] X. Lyu, C. Ren, W. Ni, H. Tian, R. P. Liu, and E. Dutkiewicz, "Optimal online data partitioning for geo-distributed machine learning in edge of wireless networks," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 10, pp. 2393–2406, 2019.

[30] D. Merkel *et al.*, "Docker: lightweight linux containers for consistent development and deployment," *Linux j*, vol. 239, no. 2, p. 2, 2014.

[31] N. Naik, "Building a virtual system of systems using docker swarm in multiple clouds," in *2016 IEEE International Symposium on Systems Engineering (ISSE)*. IEEE, 2016, pp. 1–3.

[32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[33] E. Gabriel, G. E. Fagg, G. Bosilca, T. Angskun, J. J. Dongarra, J. M. Squyres, V. Sahay, P. Kambadur, B. Barrett, A. Lumsdaine *et al.*, "Open mpi: Goals, concept, and design of a next generation mpi implementation," in *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*. Springer, 2004, pp. 97–104.

[34] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[35] Z. Ye, L. Chen, R. Lai, Y. Zhao, S. Zheng, J. Shao, B. Hou, H. Jin, Y. Zuo, L. Yin *et al.*, "Accelerating self-attentions for llm serving with flashinfer," *URL https://flashinfer. ai/2024/02/02/introduce-flashinfer. html*, 2024.

[36] "Nvidia collective communications library (nccl)," https://developer.nvidia.com/nccl.

[37] A. Tirumala, "Iperf: The tcp/udp bandwidth measurement tool," *http://dast. nlanr. net/Projects/Iperf/*, 1999.

[38] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1073–1083. [Online]. Available: https://www.aclweb.org/anthology/P17-1099

[39] D.-J. Han, D.-Y. Kim, M. Choi, D. Nickel, J. Moon, M. Chiang, and C. G. Brinton, "Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks," *IEEE Transactions on Mobile Computing*, 2023.

[40] "Sharegpt," https://huggingface.co/datasets/Aeala/ShareGPT_Vicuna_unfiltered, 2023.

[41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "vllm: Easy, fast, and cheap llm serving with pagedattention," *See https://vllm. ai/(accessed 9 August 2023)*, 2023.

[42] X. Shen, Y. Liu, H. Liu, J. Hong, B. Duan, Z. Huang, Y. Mao, Y. Wu, and D. Wu, "A split-and-privatize framework for large language model fine-tuning," *arXiv preprint arXiv:2312.15603*, 2023.

[19] H. Touvron、L. Martin、K. Stone、P. Albert、A. Almahairi、Y. Babaei、N. Bashlykov、S. Batra、P. Bhargava、S. Bhosale *et al.*，“Llama 2：开放基础和微调聊天模型”，*arXiv preprint arXiv:2307.09288*，2023 年。[20] G.-I. Yu, J. S. Jeong, G.-W. Kim、S. Kim 和 B.-G。Chun，“Orca：基于 {Transformer} 生成模型的分布式服务系统”，*16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*，2022 年，第 521-538 页。[21] A. Agrawal、N. Kedia、A. Panwar、J. Mohan、N. Kwatra、B. S. Gulavani、A. Tumanov 和 R. Ramjee，“利用 sarathi-serve 控制 llm 推理中的吞吐量-延迟权衡”，*arXiv preprint arXiv:2403.02310*，2024 年。[22] C. Holmes、M. Tanaka、M. Wyatt、A. A. Awan, J. Rasley, S. Rajb-handari, R. Y. Aminabadi, H.qin, A. Bakhtiari, L. Kurilenko *et al.*，“Deepspeed-fastgen：通过 mii 和 deepspeed-inference 实现 llms 的高吞吐量文本生成”，*arXiv preprint arXiv:2401.08671*，2024 年。[23] Y. Leviathan、M. Kalman 和 Y. Matias，“Fast通过推测性解码从 Transformer 进行推断”，在 *International Conference on Machine Learning* 中。PMLR，2023，第 19 274–19 286 页。[24] Y. Li、F. Wei、C. Zhu 和 H. Zhu，“Eagle：推测性采样需要重新思考特征不确定性”，*arXiv preprint arXiv:2401.15077*，2024 年。[25] T. Cai、Y. Li、Z. Geng、H. Peng、J. D. Lee、D. Chen 和 T. Dao，“Medusa：具有多个解码头的简单 llm 推理加速框架”，*arXiv preprint arXiv:2401.10774*，2024 年。[26] W.-L。Jiang, Z. Li, Z. Lin, Y. Shen, Z. Wu, H. Zhang, L. Cheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*，“Vicuna：一款开源聊天机器人，以 90%* chatgpt 质量打动 gpt-4”，*See https://vicuna. lmsys. org (accessed 14 April 2023)*，卷。2、没有。3，第 3 页。2023 年 6 月。[27] H. Xia、Z. Yang、Q. Dong、P. Wang、Y. Li、T. Ge、T. Liu、W. Li 和 Z. Sui，“解锁大语言模型推理的效率：推测性解码的综合综述”，见 *Findings of the Association for Computational Linguistics ACL 2024*，L.-W. 6。Ku，A. Martins 和 V. Srikumar，编辑。泰国曼谷和虚拟会议：计算语言学协会，2024 年 8 月，第 7655–7671 页。[在线的]。可用：https://aclanthology.org/2024.findings-acl.456 [28] D. Leroy、A. Coucke、T. Lavril、T. Gisselbrecht 和 J. Dureau，“关键字识别的联合学习”，参见 *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*。IEEE，2019 年，第 6341–6345 页。[29] X. Lyu、C. Ren、W. Ni、H. Tian、R. P. Liu 和 E. Dutkiewicz，“无线网络边缘地理分布式机器学习的最佳在线数据分区”，*IEEE Journal on Selected Areas in Communications*，卷。37、没有。10，第 2393–2406 页，2019 年。

[30] D. Merkel *et al.*，“Docker：用于一致开发和部署的轻量级 Linux 容器”，*Linux j*，卷。239，没有。2，第 14 页。2014 年 2 月。[31] N. Naik，“在多个云中使用 docker swarm 构建系统的虚拟系统”，参见 *2016 IEEE International Symposium on Systems Engineering (ISSE)*。IEEE，2016 年，第 1-3 页。[32] A. Paszke、S. Gross、F. Massa、A. Lerer、J. Bradbury、G. Chanan、T. Killeen、Z. Lin、N. Gimelshein、L. Antiga *et al.*，“Pytorch：命令式高性能深度学习库”，*Advances in neural information processing systems*，卷。2019 年 32 月。[33] E. Gabriel、G. E. Fagg、G. Bosilca、T. Angskun、J. J. Dongarra、J. M. Squyres、V. Sahay、P. Kambadur、B. Barrett、A. Lumsdaine *et al.*，“开放 mpi：下一代 mpi 实现的目标、概念和设计”，*Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*。施普林格，2004 年，第 97-104 页。[34] T. Dao，“Flashattention-2：更快的注意力，更好的并行性和工作划分”，*arXiv preprint arXiv:2307.08691*，2023。[35] Z. Ye，L. Chen，R. Lai，Y. Zhu，S. Cheng，J. Shao，B. Hou，H. Jin，Y. Zuo，L. Yin *et al.*，“加速自我注意力以实现 llm 服务flashinfer，” *URL https://flashinfer. ai/2024/02/02/introduce- flashinfer. html*，2024 年。[36]“Nvidia 集体通信库 (nccl)”，https://developer.nvidia.com/nccl。[37] A. Tirumala，“Iperf：tcp/udp 带宽测量工具”，*http://dast. nlanr. net/Projects/Iperf/*，1999。[38] A. See、P. J. Liu 和 C. D. Manning，“开门见山：用指针生成器网络进行总结”，见 *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*。加拿大温哥华：计算语言学协会，2017 年 7 月，第 1073–1083 页。[在线的]。可用：https://www.aclweb.org/anthology/P17-1099 [39] D.-J.汉，D.-Y。Kim、M. Choi、D. Nickel、J. Moon、M. Jiang 和 C. G. Brinton，“无线边缘网络中推理阶段优化的联合个性化泛化联合分割学习”，*IEEE Transactions on Mobile Computing*，2023 年。[40]“Sharegpt”，https://huggingface.co/datasets/Aeala/ShareGPT Vicuna unfiltered，2023 年。[41] W. Kwon, Z. Li, S. Zhuang, Y. Shen, L. Cheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “vllm: Easy, fast, and Cheap llm services with pagedattention,” *See https://vllm. ai/(accessed 9 August 2023)*, 2023. [42] X. Shen, Y. Liu, H. Liu, J. Hong, B. Duan, Z. Huang, Y. Mao, Y. Wu, and D. Wu，“用于大语言模型微调的分割和私有化框架”，*arXiv preprint arXiv:2312.15603*，2023 年。